QZPaper Huggingface Daily

2026-03-12

---

1. **OpenClaw-RL：通过对话即可训练任意智能体**
核心问题：智能体执行动作后的下一状态信号蕴含评价和指导信息被现有系统丢弃。方法：通过 Binary RL 恢复评价信号为过程奖励，通过事后启发式蒸馏恢复指导信号为 token 级监督。贡献：实现流无关的通用训练框架，单一策略可从对话、工具、GUI 等多源信号同时学习优化。
2. **Flash-KMeans：快速且内存高效的精确 K-Means 实现**
核心问题：K-means GPU 实现性能被内存 IO 而非计算复杂度限制。方法：提出 IO 感知流水线，融合赋值与更新避免矩阵物化和冗余访问。贡献：相比最优基线最高加速 33 倍，支持十亿数据点高效处理，提升大规模聚类性能。
3. **MA-EgoQA：面向多具身智能体第一人称视频的问答系统**
核心问题：多具身智能体系统需跨长视频流整合事件回答关于多智能体交互的问题。方法：提出 MA-EgoQA 基准含五类多智能体场景，采用单、多时间段混合生成策略构建问答对。贡献：首次定义多智能体长视频理解任务，含 1741 个跨视角问答对覆盖社交协调、推理等维度。
4. **ReMix：基于强化学习路由的混合 LoRA 高效微调方法**
核心问题：混合 LoRA 结构中路由权重坍塌，多个被激活的 LoRA 实际贡献有限。方法：采用强化学习动态路由，通过有效支撑大小度量约束 LoRA 激活分布平衡。贡献：理论证明权重坍塌现象，提出 ReMix 方法实现多 LoRA 的均衡有效利用与性能提升。
5. **ICRL：通过上下文强化学习实现大语言模型的工具使用**
核心问题：工具使用训练需大量标注工具轨迹或 SFT 预热，效率低成本高。方法：纯 RL 框架通过在 rollout 中嵌入少量演示代替 SFT，逐步减少示例实现过渡。贡献：消除标注依赖使模型从模仿过渡到自主工具使用，提高新工具适应效率。
6. **OAKS：大语言模型能否跟上持续知识流的在线适应?**
核心问题：现有基准未充分评估 LLM 在动态知识流下的在线适应能力与持续学习。方法：提出 OAKS 基准，事实流式顺序到达可更新或矛盾，模型需动态修正知识。贡献：首个统一持续学习与在线适应基准，含 BABILong 和小说数据集，覆盖稀疏至频繁更新。
7. **LLM2VEC-GEN：基于大语言模型的生成式嵌入表示**
核心问题：嵌入模型输入中心范式面临输入-输出鸿沟，需大量标注配对数据。方法：创新编码 LLM 对输入的潜在响应，仅需无标注查询与冻结模型加轻量可训参数。贡献：自监督 SOTA 性能、有害内容检索降低 43%、推理密集提升 29%，嵌入具自然语言解释性。
8. **RetroAgent：通过回顾式双重内在反馈实现从求解到进化**
核心问题：标准 RL 偏重静态求解忽视持续适应，智能体探索不足且学到知识隐式化。方法：事后自我反思产生数值和语言双重内在反馈，驱动策略优化与记忆积累。贡献：统一框架实现求解向持续适应的进化，提升探索覆盖与知识复用，增强泛化性能。
9. **CodePercept：基于代码锚定的 MLLM 视觉 STEM 感知增强**
核心问题：STEM 视觉推理中感知不足是瓶颈，自然语言无法完整捕获复杂空间关系。方法：以可执行代码作为感知锚定，通过代码复现和多样化扩展构建大规模图像-代码对。贡献：缩放分析证明感知关键性，代码锚定增强视觉理解，ICC-1M 数据集支撑高效训练。
10. **PRISM-$Δ$ *Prompt***
核心问题：提示高亮方法仅操作路由通道（Key），忽视内容通道（Value）的关键信号。方法：差分交叉协方差分解同时引导两通道，捕获路由增益、内容增益和交叉增益。贡献：双通道注意力调整方法增强目标信息关注度，显著改进长文本与冲突事实处理能力。

OpenClaw-RL：通过对话即可训练任意智能体

Wang*, Chen*, Jin*, Wang†, Yang† — Gen-Verse, 2026

1. 核心思想与动机

每个已部署的 AI 智能体在执行动作 a_t 后都会收到一个**下一状态信号** s_{t+1}（用户回复、工具执行结果、GUI 状态变化、测试结果等），但现有系统仅将其作为下一步动作的上下文，**丢弃了其中蕴含的两类宝贵训练信号**：

- **评价性信号（Evaluative）**：隐式评分前一动作——用户重新提问表示不满，测试通过表示成功，错误日志表示失败。这构成天然的过程奖励，无需额外标注；
- **指导性信号（Directive）**：指明动作应如何改变——用户说"你应该先检查文件再编辑"不仅表明回复有误，还指出了哪些 *token* 应被修改及如何修改。标量奖励完全无法捕获此信息。

OpenClaw-RL 的核心洞察：**下一状态信号是流无关的（stream-agnostic）**，个人对话、终端执行、GUI 交互、SWE 任务和工具调用都产生此类信号，同一策略可在同一训练循环中从所有信号同时学习。系统通过 Binary RL 恢复评价性信号为标量过程奖励，通过 Hindsight-Guided On-Policy Distillation（OPD）恢复指导性信号为 token 级方向性优势监督，两者互补结合实现显著优化增益。

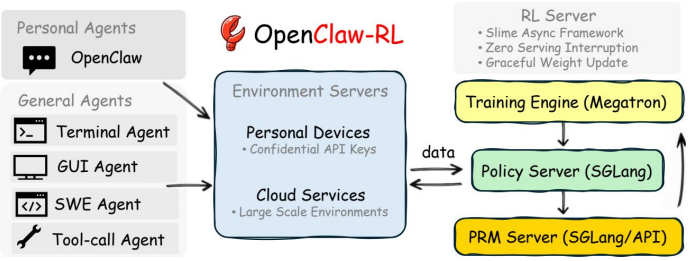


图 1：*OpenClaw-RL* 基础设施概览。个人智能体与通用智能体的交互流统一接入基于 *slime* 的异步 *RL* 服务器。

2. 问题形式化

每个交互流建模为 MDP $(S, \mathcal{A}, \mathcal{T}, r)$：状态 s_t 为完整对话/环境上下文；动作 a_t 为策略 π_θ 生成的 token 序列；转移 $\mathcal{T}(s_{t+1} \mid s_t, a_t)$ 由环境确定性给出；奖励 $r(a_t, s_{t+1})$ 通过 PRM 法官从下一状态信号推断。关键区别于标准 RLVR：过程奖励 $r(a_t, s_{t+1})$ 依赖 s_{t+1}，包含远比终局结果奖励丰富的信息。当 s_{t+1} 包含显式指导信息时，OPD 进一步将其转化为 token 级教师监督。

3. 系统架构

3.1 四组件全解耦异步流水线

OpenClaw-RL 基于 slime 构建，核心设计原则为**完全解耦**：策略服务（SGLang）、环境托管（HTTP/API）、PRM 评判（SGLang/API）、策略训练（Megatron）作为四个独立异步循环运行，**无阻塞依赖**。模型在服务下一个用户请求的同时，PRM 评判前一个回复，训练器应用梯度更新——三者互不等待。这使从实时异构交互流持续训练成为可能。

对**个人智能体**，模型通过机密 API 连接用户设备，无需修改智能体框架，权重优雅更新不中断推理。对**通用智能体**，异步设计允许各组件独立推进，缓解长时域 rollout 的长尾问题。

3.2 个人智能体的会话感知环境服务器

每个 API 请求分为两类：**主线轮次**（智能体主回复与工具执行结果，构成可训练样本）和**辅助轮次**（辅助查询、记忆组织等，转发但不产生训练数据）。每个新主线请求的消息包含对前一轮次的反应（用户回复或执行结果），即为前一轮次的 s_{t+1}。仅主线轮次参与训练。

3.3 可扩展性与多场景支持

系统覆盖从单用户个性化到大规模多环境部署的全谱段。通用智能体支持五类场景：

场景	环境	下一状态信号
OpenClaw	个人设备	用户回复/工具结果
Terminal	Shell 沙箱	stdout/stderr, exit code
GUI	屏幕 + 可访问性树	视觉状态差异
SWE	代码仓库 + 测试	测试结果, diff, lint
Tool-call	API/函数执行	返回值, 错误追踪

云托管环境支持大规模并行化（Terminal 128 个、GUI/SWE 64 个、Tool-call 32 个并行环境）。所有交互和奖励评估实时记录至 JSONL，日志写入为非阻塞后台线程，不增加服务延迟。

4. 学习方法

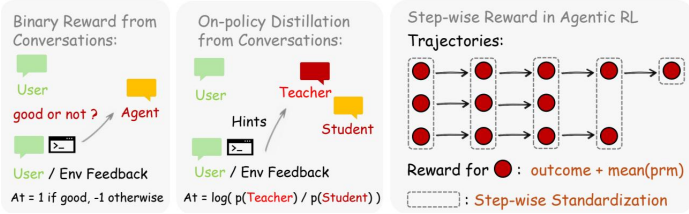


图 2：方法概览。个人智能体同时支持 *Binary RL* 和 *OPD* 训练；通用智能体集成过程与结果奖励。

4.1 Binary RL：评价性信号 → 标量过程奖励

PRM 法官构建：给定回复 a_t 和下一状态 s_{t+1}，法官模型评估质量：$\mathrm{PRM}(a_t, s_{t+1}) \to r \in \{+1, -1, 0\}$。运行 m 次独立查询，取多数投票 $r_{\text{final}} = \mathrm{MajorityVote}(r_1, \ldots, r_m)$。

训练目标：直接使用 $A_t = r_{\text{final}}$ 作为优势，采用**非对称 PPO 裁剪代理**：

$$\rho_t = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}, \quad \mathcal{L}_{\text{pg}} = -\mathbb{E}_t\Big[\min\big(\rho_t A_t,\ \mathrm{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon_{\text{high}}) A_t\big)$$

$$\mathcal{L} = \mathcal{L}_{\text{pg}} + \beta_{\text{KL}} \cdot \mathcal{L}_{\text{KL}}$$

其中 $\varepsilon = 0.2$，$\varepsilon_{\text{high}} = 0.28$，$\beta_{\text{KL}} = 0.02$。实时对话场景无 group 结构，因此不能使用 GRPO 式标准化。

4.2 Hindsight-Guided OPD：指导性信号 → Token 级监督

Binary RL 将 s_{t+1} 的全部信息压缩为单一标量。OPD 通过四步恢复 token 级方向性信号：

步骤 1 —Hindsight Hint 提取：法官从 s_{t+1} 提取简洁可操作的修正提示（1–3 句），关键设计：不直接使用原始 s_{t+1}（通常含噪声/无关内容），而是蒸馏出纯粹的指导性内容。

步骤 2 —Hint 选择与质量过滤：在正面投票中选最长 hint（>10 字符）。无有效 hint 则丢弃样本——OPD 以样本量换信号质量，仅含明确修正方向的轮次进入训练。

步骤 3 —增强教师构建：将 hint 附加到原始提示：$s_{\text{enhanced}} = s_t \oplus \text{hint}$，构造模型"若用户预先提供修正"时看到的提示。

步骤 4 —Token 级优势计算：在 s_{enhanced} 下以原始回复 a_t 为强制输入查询模型：

$$A_t = \log \pi_{\text{teacher}}(a_t \mid s_{\text{enhanced}}) - \log \pi_\theta(a_t \mid s_t)$$

$A_t > 0$：教师（知道 hint）对该 token 赋更高概率，学生应增强；$A_t < 0$：教师认为该 token 不当，学生应抑制。**同一回复内不同 token 可被不同方向调整**，这是标量奖励无法实现的。

4.3 Binary + OPD 联合训练

两方法互补而非竞争。Binary RL 接受所有评分轮次（广覆盖、粗信号），OPD 仅在含丰富指导内容的子集上提供高分辨率 token 级修正。联合优势：

$$A_t = w_{\text{binary}} \cdot r_{\text{final}} + w_{\text{opd}} \cdot \big(\log \pi_{\text{teacher}}(a_t \mid s_{\text{enhanced}}) - \log \pi_\theta(a_t \mid s_t)\big)$$

默认 $w_{\text{binary}} = w_{\text{opd}} = 1$。两方法共享同一 PPO 损失，仅优势计算不同。

4.4 通用智能体的逐步奖励

长时域任务中仅终局奖励使绝大多数轮次无监督。遵循 RLAnything，将结果奖励与过程奖励简单相加：步骤 t 的奖励为 $o + \sum_{i=1}^{m} r_i/m$，其中 r_i 由 $\mathrm{PRM}(a_t, s_{t+1})$ 独立赋予。对优势标准化，直接按**步索引分组**（因实际环境中状态难以聚类），实验验证有效。

5. 实验结果

5.1 个人智能体：从对话信号学习个性化

模拟两个场景：**学生**使用 OpenClaw 做作业（不想被发现用 AI，要求自然风格）；**教师**使用 OpenClaw 批改作业（要求评语具体友好）。策略模型为 Qwen3-4B，学习率 1×10^{-5}，每 16 个样本触发一次训练。

方法	8 步后	16 步后
基线	0.17	
Binary RL	0.25	0.23
OPD	0.25	0.72
Combined	**0.76**	**0.81**

关键发现 [Q1]：联合方法最优。OPD 因训练样本稀疏而延迟生效（8 步时与 Binary RL 相当，16 步时显著超越）；Binary RL 单独仅带来边际提升。

关键发现 [Q2]：学生场景仅需 36 次交互、教师场景仅需 24 次即实现显著可见改善。学生场景中，智能体学会避免明显 AI 风格（加粗、过度结构化），转向自然随意风格；教师场景中，从冷淡简短（"Correct. Well done!"）转为具体友好的详细反馈。基线分数：学生 0.17→0.76，教师 0.22→0.90。

5.2 通用智能体：跨场景统一 RL

使用不同规模模型验证框架通用性：Terminal (Qwen3-8B, SETA)、GUI (Qwen3VL-8B-Thinking, OSWorld)、SWE (Qwen3-32B, SWE-Bench-Verified)、Tool-call (Qwen3-4B-SFT, DAPO→AIME 2024)。学习率 10^{-6}，$\beta_{\mathrm{KL}} = 0.01$，每任务 8 个采样。

关键发现 [Q3]：框架成功处理所有四类真实场景，支持不同模型规模（4B–32B）和模态（文本/多模态），大规模环境并行化进一步提升训练可扩展性。四个场景的 rollout 准确率/评估分数均随 RL 步数稳步提升。

关键发现 [Q4] —过程奖励至关重要：

场景	集成奖励	仅结果奖励
Tool-call (250 步)	**0.30**	0.17
GUI (120 步)	**0.33**	0.31

Tool-call 场景中集成过程奖励带来 76% 相对提升（0.17 → 0.30），GUI 场景也有 6.5% 提升。代价是需额外资源托管 PRM。

6. 与相关工作的对比

vs. RLHF/DPO/GRPO：这些方法均为批处理离线模式（数据收集与训练分离），OpenClaw-RL 从实时交互信号持续训练，无需数据预收集阶段。

vs. 特定场景 Agent RL（DigiRL、WebRL、SWE-agent 等）：各自针对单一环境、使用专用流水线。OpenClaw-RL 以统一基础设施覆盖个人对话 + 四类通用场景。

vs. 在线蒸馏方法（Agarwal 等, Hübotter 等）：依赖预收集的反馈-回复对。OPD 从实时下一状态信号提取 hint，模型自身作为教师（自蒸馏），无需外部教师、无需配对偏好数据。

vs. RL 训练基础设施（OpenRLHF、AReal、veRL、slime）：均假设批处理数据收集。OpenClaw-RL 在 slime 之上实现四组件全异步，支持从实时多流交互持续训练且零中断服务。

7. 局限性与未来方向

- 个人智能体实验基于 LLM 模拟用户，真实用户交互的噪声与稀疏性更具挑战；
- OPD 依赖下一状态中包含可提取的指导性内容，对隐式/模糊反馈效果有限；
- PRM 法官本身引入额外计算开销与潜在判断偏差；
- 当前过程奖励按步索引分组标准化，对复杂非齐次轨迹可能不够精细；
- 个人智能体的隐私与安全（机密 API 传输训练数据）需进一步验证；
- 未来方向包括：扩展至更大规模模型、探索 OPD 在通用智能体场景的应用、以及开发更精细的过程奖励标准化策略。

Flash-KMeans：快速且内存高效的精确 K-Means 实现

Yang*, Xi*, Zhao, Li, Fan, Zhang, Cai, Lin, Li, Keutzer, Han, Xu, Stoica — UC Berkeley / UT Austin / MIT / NVIDIA, 2026

1. 动机与问题背景

k-means（Lloyd, 1982）正经历从离线分析工具到**在线高频算子**的范式转变：向量量化检索（ColBERT）、稀疏注意力路由（Routing Transformer, TACTIC）、KV cache 压缩（ClusterKV, KVQuant）、视频扩散模型语义感知 token 置换（SparseVideoGen）等场景均需在训练/推理流水线中反复调用 k-means。这带来三维需求升级：（1）执行从 CPU 迁移至 GPU；（2）从离线转为在线，要求严格低延迟；（3）序列更长、batch 更大，要求高吞吐与可扩展性。

然而，现有 GPU 实现（cuML、FAISS、fast_pytorch_kmeans）的端到端性能被**底层系统约束**而非算法复杂度所主导。传统算法优化（三角不等式剪枝、coreset 采样等）虽减少理论 FLOPs，但在现代 GPU 上 dense compute 极其廉价（Tensor Core 矩阵乘），而碎片化内存操作才是真正瓶颈。本文提出 **flash-kmeans**，一种 IO 感知、无竞争的精确 k-means 实现，不改变 *Lloyd* 算法的数学公式，仅重构执行数据流以匹配硬件约束。

指标	关键结果
端到端最大加速	17.9×（vs. best baseline）
vs. cuML / FAISS	33× / > 200×
FlashAssign 内核加速	最高 21.2×
Sort-Inverse Update 内核加速	最高 6.3×
Out-of-core（10 亿点）	10.5× 加速
编译调优开销削减	175×（性能损失 < 0.3%）

2. 标准实现的瓶颈分析

2.1 Lloyd 算法回顾

给定 $X \in \mathbb{R}^{N\times d}$，质心 $C \in \mathbb{R}^{K\times d}$，最小化 $\min_{a,C}\sum_{i=1}^{N}\|x_i - c_{a_i}\|_2^2$。每次迭代分两阶段：**赋值阶段**计算距离矩阵 $D_{ik} = \|x_i - c_k\|_2^2$，$a_i = \arg\min_k D_{ik}$；**质心更新阶段**聚合 $n_k = \sum_i \mathbb{I}[a_i = k]$，$s_k = \sum_i \mathbb{I}[a_i = k]x_i$，$c_k \leftarrow s_k/n_k$。实际中距离展开为 $\|x_i\|^2 + \|c_k\|^2 - 2x_i^\top c_k$ 以复用范数并利用矩阵乘。

2.2 赋值阶段的 IO 瓶颈

标准实现先计算距离写入 HBM，再读回做 argmin，产生 $2\cdot\Theta(NK)$ 次 HBM 读写。在 $N = 65536, K = 1024, d = 128, B = 32$ 的代表性设置下，距离计算仅需 2.6ms，而由 D 矩阵物化引发的内存时间高达 23ms——IO 开销是计算的近 9×。输入读取仅 $\Theta(Nd + Kd)$，输出仅 $\Theta(N + Kd)$，D 的物化完全主导带宽。

2.3 更新阶段的原子写竞争

标准实现以 token 粒度执行 scatter-style atomic add：每个线程读取 x_i 和 a_i，原子累加到全局 sum/count 缓冲区。当多个 warp 同时更新同一"热"质心时，导致严重的原子竞争、序列化和 cache-line thrashing。原子操作数量 $O(Nd)$。在 H200 GPU 上实测有效带宽仅 **50 GB/s**，远低于常规规约的硬件理论带宽。

2.4 系统级约束

（1）大 batch 输入超 VRAM 容量时，分块执行引入 PCIe 通信开销；（2）动态 shape 需反复自动调优，编译开销严重影响 time-to-first-run。

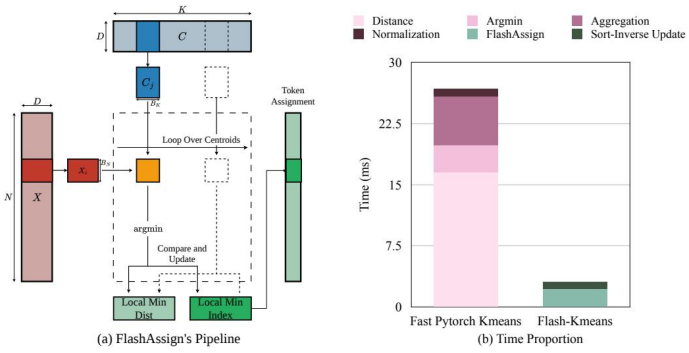


图 1：*flash-kmeans* 概览与性能分解。*(a) FlashAssign* 流式处理数据块，融合距离计算与在线 *argmin*，完全绕过 $N\times K$ 距离矩阵物化。*(b)* 相比标准实现，赋值 *IO* 瓶颈和更新同步瓶颈被大幅压缩。

3. 核心方法

3.1 FlashAssign：无物化赋值

灵感源自 FlashAttention 的 IO 感知思想。对每个点 x_i 维护寄存器中的**在线 argmin 状态**：当前最小距离 m_i（初始 $+\infty$）和对应质心索引 a_i（初始 -1）。质心按 tile 扫描，每个 tile 在片上计算局部距离、求局部最小值 $(\tilde{m}, \tilde{a})$，与运行状态比较保留更小者。遍历所有 tile 后得到精确全局最小值。

Tiling 与异步预取。采用二维 tiling（点 tile B_N × 质心 tile B_K），双缓冲异步预取：加载下一质心 tile 与当前 tile 距离计算重叠执行，隐藏内存延迟。

IO 复杂度。融合后每次赋值仅需读 X 和 C 各一次、写赋值向量 a 一次，IO 复杂度从 $O(NK)$ 降至 $O(Nd + Kd)$，**完全消除** $2\cdot\Theta(NK)$ 的 HBM 往返开销。

3.2 Sort-Inverse Update：低竞争质心聚合

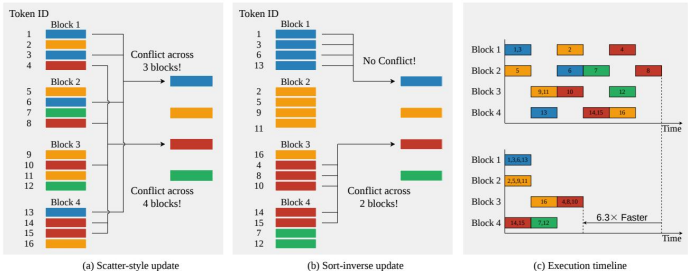


图 2：质心更新对比。*(a)* 标准 *scatter* 更新：不规则映射导致严重原子竞争。*(b) Sort-inverse update*：排序后相同 *cluster id* 自然连续，转化为 *segment* 级局部规约。*(c)* 时间线：标准方法频繁因原子锁停顿，*sort-inverse* 实现无竞争内存写入。

核心思想：将 token-to-cluster scatter 变换为 cluster-to-token gather。

步骤 1——显式逆映射构建：对赋值向量 a 执行 argsort 得到排列索引 `sorted_idx`，构造排序后的 cluster-id 序列 a^{sorted}。相同 cluster id 自然形成连续 segment。注意：仅排序 *1D* 赋值向量，不物理搬移重量级点矩阵 X。

步骤 2——Segment 级局部聚合：每个 CTA 处理 a^{sorted} 的连续块，识别 segment 边界，通过 `sorted_idx` 从原始 X 中 gather 对应特征。部分和与计数**完全在片上**（寄存器/共享内存）累积，仅在 segment 边界执行一次全局 `atomic_add`。读侧为常规 HBM gather，写侧规约完全在片上完成，绕过写路径瓶颈。

原子操作数分析。排序后序列恰含 K 个连续 segment，CTA 分块边界最多额外增加 $\lceil N/B_N\rceil$ 个断点。最坏情况原子合并数从 $O(Nd)$（标准）降至 $O((K + \lceil N/B_N\rceil)d)$，数量级缩减直接消除写竞争。

3.3 算法-系统协同设计

分块流式重叠（Chunked Stream Overlap）。当输入超 VRAM 容量时，数据分块通过 CUDA stream 协调异步 Host-to-Device 传输与计算：当前块计算时并行传输下一块，双缓冲流水线模式。有效隐藏 PCIe 通信开销，支持扩展至**十亿点**规模。

Cache 感知编译启发式。根据硬件特征（L1/L2 cache 大小）和问题 shape 解析式推导内核配置，替代昂贵的穷举自动调优。目标非完美调优每个 shape，而是以极低编译开销可靠匹配近最优配置。

4. 实验评估

在 **NVIDIA H200 GPU**（CUDA 12.8）上系统评估，全面扫描 N, K, D, B 参数空间，对比四个高度优化基线：fast_pytorch_kmeans、fastkmeans、NVIDIA cuML、FAISS。

4.1 端到端加速

按三个代表性场景分组：

- **大 N 大 K**（$N = 1\text{M}, K = 64\text{K}, D = 512$）：标准 PyTorch 因 $N\times K$ 距离矩阵 OOM；flash-kmeans 比最强基线

（fastkmeans）加速 $> 5.4\times$；

- **大 N 小 K**（$N = 8\text{M}, K = 1024$）：计算密集型，flash-kmeans 延迟降低 94.4%，加速 **17.9**×（vs. fast_pytorch_kmeans）；
- **小 N 小 K**：内核启动开销主导，flash-kmeans 在高 batch（$B = 32$）下仍达 15.3× 加速。

对比工业标准库：vs. cuML 加速 **33**×，vs. FAISS 加速 $>$ **200**×。flash-kmeans 在所有 batch 大小和特征维度下均保持鲁棒优势。

4.2 内核级分解

FlashAssign：在高压配置（$N = 1\text{M}, K = 8192$）下，标准赋值耗时 122.5ms，FlashAssign 仅 5.8ms，加速 **21.2**×。

Sort-Inverse Update：在大规模配置（$B = 1, N = 33\text{M}, K = 4096$）下加速 **6.3**×。微基准确认两大瓶颈（IO 限制与同步瓶颈）均被有效消除。

4.3 算法-系统协同设计评估

大规模 Out-of-core 处理。标准 PyTorch 在超 VRAM 场景直接 OOM。与最鲁棒的 out-of-core 基线 fastkmeans 对比：

- $N = 10^9, K = 32768, D = 128$：flash-kmeans 41.4s vs. 基线 261.8s，加速 6.3×；
- $N = 400\text{M}, K = 16384$：flash-kmeans 8.4s vs. 基线 88.4s，加速 **10.5**×。

流水线执行成功限制 GPU 峰值内存占用，同时实现数量级加速。

动态 shape 快速部署。穷举调优在大 shape（$N = 8\text{M}, K = 65536$）下需 $>$325s；cache 感知启发式在所有测试 shape 上仅需 $<$2.5s，调优开销削减 **175**×。关键是执行性能无损：启发式驱动的内核延迟与穷举最优差异 $<$ **0.3**%，实现开箱即用的高性能部署。

5. 与相关工作的定位

flash-kmeans 延续了 **FlashAttention** 开创的 IO 感知优化范式——不改变数学公式，而是重构算子内存数据流以匹配硬件带宽/同步约束。FlashAttention 消除 $N \times N$ 注意力矩阵物化，flash-kmeans 类似地消除 $N \times K$ 距离矩阵物化。Sort-inverse update 则借鉴了 Flash-Decoding 和 SonicMoE 中将不规则 scatter 写转化为排序后 segment 规约的策略。与算法级优化（三角不等式、coreset）不同，flash-kmeans 聚焦实现级优化，两者正交互补——算法 FLOP 缩减需以高效实现为前提才能转化为真实加速。

6. 局限性与未来方向

- 当前仅支持**欧氏距离**，余弦距离等其他度量的融合策略需进一步设计；
- FlashAssign 的 tiling 策略在 K 极大（$>$100K）且 d 极高时，片上缓存压力增大，可能需更精细的多级 tiling；
- Sort-inverse update 依赖高效 argsort，排序本身在极端 N 下有非忽略开销（尽管远小于原子竞争代价）；
- Cache 感知启发式针对 NVIDIA GPU 设计，跨平台（AMD、自研加速器）适配需扩展硬件模型；
- 未探索与算法级优化（mini-batch、三角不等式）的组合，以及与 k-means++ 初始化的端到端集成；
- 作为精确实现，可进一步探索与近似方法的精度-速度权衡空间。

MA-EgoQA：面向多具身智能体第一人称视频的问答系统

Kim[1], Yang[2], Kim[1,3], Yeo[1], Lee[1,4], Ren[2†], Hwang[1,5†] — [1]KAIST, [2]NYU, [3]Samsung, [4]ETRI, [5]DeepAuto.ai

1. 动机与问题定义

随着具身智能模型日益强大，未来人类将在工作或家庭场景中与**多个具身 AI 智能体**协同工作。为确保人类用户与多智能体系统之间的高效通信，系统需要并行解读各智能体的输入信息，并为每个查询引用恰当的上下文。然而该问题面临两大核心挑战：

- **超长视频流压缩与通信**：各智能体持续采集第一人称视频，现实中可能运行数天，每个智能体产生海量视频历史，难以定位查询相关事件。当前 Video LLM 仍仅能处理数小时级别的视频，远不满足多智能体 QA 需求；
- **多视角信息聚合**：系统需超越单智能体理解，**跨智能体整合事件**以实现系统级理解——例如查询多路执法记录仪中的异常事件、或确认家庭机器人上次清洁浴室的时间。

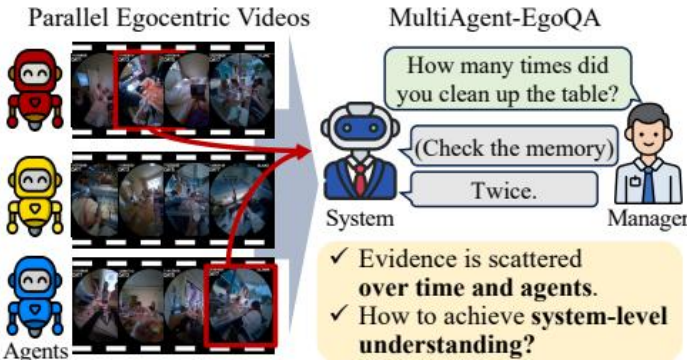


图 1：*MA-EgoQA* 问题定义与核心挑战。人类管理者向多智能体系统查询，系统需跨智能体整合经验并检索相关片段。

形式化定义：给定 N 个具身智能体，每个智能体 A_i 持续录制第一人称视频流 V_i，时长 T 小时。完整多智能体视频集合 $\mathcal{V}=\{V_1,V_2,\ldots,V_N\}$，总时长 $N\times T$ 小时。给定用户查询，系统需基于 $\mathcal{V}$ 生成回答。每个查询涉及**两个以上智能体**的信息，要求：(1) 构建跨智能体的时序与上下文对齐的全局记忆；(2) 从多智能体记忆中高效检索相关事件。

本文首次正式定义**多智能体长时第一人称视频理解问题**，提出基准 MA-EgoQA 和基线模型 EgoMAS。

2. MA-EgoQA 基准

2.1 数据基础与类别设计

MA-EgoQA 基于 EgoLife 数据集构建——6 人在共享房屋中佩戴摄像头眼镜连续 7 天采集第一人称视频（共 266 小时）。基准包含 **1,741** 个问答对，覆盖五个多智能体场景特有类别：

- **社交互动（SI, 376 题）**：评估跨视频流的非正式对话和亲和行为定位能力，关注人际互动、群体行为和对话中的关键信息；
- **任务协调（TC, 363 题）**：反映 MAS 协作的实际重要性，考察角色分配、责任划分、行动排序和决策过程；
- **心智理论（ToM, 235 题）**：推理他人心理状态（信念、意图、情感），包括智能体的误解、感知限制和行动意图；
- **时间推理（TR, 417 题）**：分为并发（同一时刻各智能体活动）和比较（跨智能体事件时序排列）两个子类；
- **环境交互（EI, 350 题）**：追踪物体使用频率、首次使用、最多使用者等，需聚合多智能体的环境交互信息。

SI 和 TC 类别额外设计**多时间段**（multi-span）问题，需跨多个不连续时间窗口推理，使基准真正具备长时推理特性。所有问题均为**五选一**选择题。

2.2 数据生成管线

采用**两种互补策略**生成 QA 对：

单时间段生成：将每个智能体的长视频划分为 5 分钟段，收集该时段内所有智能体的密集描述和转录文本，提供给 GPT-4o 生成问答对（含问题、答案、理由、引用的智能体与时间戳、4 个错误选项）。每个问题须基于 ≥ 2 个智能体。共生成 SI/TC/ToM 类别各 33.4k/31.6k/34.1k 个候选样本。

多时间段生成：通过文本编码器 $E(\cdot)$ 获取句子嵌入 $e=E([q;a])$，计算余弦相似度 $s_{ij}=\langle e_i,e_j\rangle/\|e_i\|\|e_j\|$，构建相似度图（阈值 δ），提取连通分量作为语义分组，由 GPT-5 合成跨时间段问答（SI 生成 15.9k，TC 生成 16.3k）。

模板生成：TR 和 EI 类别使用预定义模板。TR 在 30 秒/10 分钟/1 小时三个时间尺度生成描述；EI 手动列出共享环境中的交互物体，收集相关描述后填充模板。

2.3 LLM 过滤与人工验证

四阶段质量控制：**(1) 零样本过滤**——无上下文时 GPT-5 三次中 >2 次答对则排除；**(2) 单智能体过滤**——仅使用单人记忆即可回答的样本排除（确保需多智能体信息）；**(3) 跨模型验证**——Gemini-2.5-Flash 和 Claude-Sonnet-4 交叉检验答案正确性和选项质量；**(4) 人工审核**——4 名标注者从 3,436 个候选中选出 1,741 个高质量样本。

3. EgoMAS 基线模型

EgoMAS 是一个**无训练**的集中式多智能体系统，包含两个核心组件：

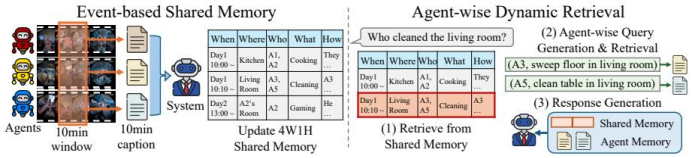


图 2：*EgoMAS* 概览。构建基于事件的共享记忆，接收查询后动态检索相关智能体信息。

3.1 基于事件的共享记忆

每 10 分钟各智能体提供观测摘要，**中央管理器**整合为系统级摘要。区别于扁平文本压缩，管理器识别跨智能体关键事件，为每个事件显式记录 **4W1H 字段**（When、What、Where、Who、How），生成对齐智能体视角且保留推理关键细节的全局记忆。

3.2 智能体级动态检索

给定查询 q，检索分两阶段。**第一阶段**：从共享记忆 $\mathcal{M}_{\text{shared}}$ 中 BM25 检索 top-n 系统级记忆：

$$R_{\text{sys}}(q)=\text{Top-}n\{(m,s(m,q))\mid m\in\mathcal{M}_{\text{shared}}\}$$

第二阶段：基于系统级上下文生成**智能体特定检索请求** $\mathcal{Q}_{\text{agent}}=\{(a_j,q_j)\}_{j=1}^{J}$，对每个智能体的记忆 $\mathcal{M}_{a_j}$ 执行 top-k 检索并以阈值 τ 过滤：

$$\widetilde{\mathcal{R}}_{a_j}(q_j)=\{(m,s(m,q_j))\in\mathcal{R}_{a_j}(q_j)\mid s(m,q_j)\geq\tau\}$$

最终响应基于系统级和聚合智能体级结果生成：$\hat{y}=F(q,R_{\text{sys}}(q),\widetilde{\mathcal{R}})$，其中 $\widetilde{\mathcal{R}}=\bigcup_{j=1}^{J}\widetilde{\mathcal{R}}_{a_j}(q_j)$。超参数设为 $n=20$，$k=5$，$\tau=10$。

4. 实验结果

4.1 主要结果

评估 16 个基线（含专有/开源 LLM、Video LLM、RAG 方法）和 EgoMAS。

模型（类别）	平均准确率
随机基线	20.00%
Gemini-2.5-Flash（全描述拼接，1M）	36.93%
GPT-5（全描述拼接，272k）	34.81%
Qwen2.5-7B-1M（全描述拼接）	26.08%
VideoChat-Flash（全帧拼接）	23.06%
VideoXL-2（全帧拼接）	20.39%
BM25（RAG）	36.01%
WorldMM-8B（RAG）	27.63%
EgoMAS (Gemini-2.5-Flash)	**41.41%**
EgoMAS (Qwen3VL-8B-Thinking)	40.26%
EgoMAS (Qwen3VL-8B-Instruct)	37.68%
Oracle (Gemini-2.5-Flash)	83.80%

关键发现：(1) 即使最强的 Gemini-2.5-Flash 仅达 36.93%，gpt-oss 和 VideoXL-2 仅略高于随机水平，凸显任务难度；

(2) 全帧拼接方法（无转录信息 + 大量无关帧）表现最差；(3) RAG 方法（BM25、WorldMM-8B）在更短输入上下文下优于非检索开源模型；(4) **EgoMAS (Gemini-2.5-Flash) 以 41.41% 超越同骨干基线 4.48%**，且 Qwen3VL-8B 级 EgoMAS 亦超越 Gemini-2.5-Flash 和 GPT-5 全描述基线；(5) 所有模型与 Oracle（83.80%）差距 ∼42%，仍有巨大提升空间。

4.2 分类别分析

ToM 是最困难类别——所有模型在 ToM 上准确率最低（EgoMAS 仅 33.62%）。因 ToM 需推理隐含心理状态（目标、信念）而非提取显式视觉/文本线索，涉及递归心智状态推理和复杂目标-信念-行动推断。

多时间段推理显著更难： SI 单段 46.30% vs. 多段 23.08%，TC 单段 43.27% vs. 多段 23.81%（EgoMAS-Qwen3VL-8B），需在 7 天视频中识别和关联多个事件。

所需智能体数量增加时性能持续下降，表明当前多智能体知识融合方法仍有限。

4.3 EgoMAS 效率分析

EgoMAS 每查询仅需 **1.3 秒**，输入上下文仅 4.6k–7.4k token（vs. Gemini 的 1M token），在检索类模型中延迟最低且准确率最高。限制为单智能体记忆时性能大幅下降，验证了多智能体记忆的必要性。

5. 消融实验

组件消融（Qwen2.5VL-7B 骨干）：无共享记忆 + 无动态检索 27.80%，仅动态检索 28.20%，仅共享记忆 30.04%，**两者结合 35.55%**——共享记忆和动态检索各自贡献显著。

记忆结构对比： 4W1H 事件记忆（35.55%）显著优于 Summary（30.67%）、Triplet（30.44%）、Chunk（25.96%）和 Graph（31.99%），表明结构化事件抽取在跨智能体信息融合上的优势。

检索器对比： NV-Embed-v2 (7B) 达最高 37.91%，但 BM25 以轻量关键词检索达 35.55% 的竞争性能（DPR 仅 28.67%，Qwen3-Embed-0.6B 33.03%），兼具实用性与效果。

超参数敏感性： 共享记忆检索量 n 在 20–60 最优（过小/过大均损害性能）；模型对 k 较鲁棒；高阈值 τ 因过度过滤而降低准确率。可用智能体数量从 1 增至 6 时，准确率从 31.99% 升至 35.55%，验证多智能体信息的必要性。

6. 与现有基准的对比

MA-EgoQA 是首个同时满足以下条件的基准：(1) 第一人称视角；(2) **跨视频流**（多智能体）；(3) **超长时间**（266 小时 vs. EgoSchema 180s、EgoMemoria 60min）；(4) 包含 ToM 推理；(5) 跨天级推理。相比之下，EgoLifeQA 仅单智能体假设，MuMA-ToM 仅第三人称且时长 36 秒，CVBench 虽支持跨视频但非第一人称且时长短。

7. 局限性与未来方向

- MA-EgoQA 仅基于 EgoLife 数据集（6 人共享房屋场景），尚未覆盖其他环境（目前是唯一公开的多人同步长期第一人称视频数据集）；
- EgoMAS 与 Oracle 差距约 42%，表明多智能体第一人称视频 QA 仍需重大突破；
- 更强的检索器可直接提升性能（NV-Embed-v2 达 37.91%），未来可探索混合检索（文本嵌入 + 词汇匹配）；
- 当前模型在 ToM 和多时间段推理上表现最差，需发展超越简单记忆共享的**全局一致性理解**能力；
- EgoMAS 性能随骨干模型能力提升而改善（Qwen2.5VL-7B → Gemini-2.5-Flash：35.55% → 41.41%），更强的推理能力支撑更有效的检索策略和多智能体信息整合。

ReMix：基于强化学习路由的混合 LoRA 高效微调方法

Qiu[1], Zeng[2], Xia[2], Feng[3], Fu[2], Liu[1], Yoo[1], Zeng[1], Wei[1], Tong[1] 等 — [1]UIUC, [2]Meta AI, [3]WashU, 2025

1. 问题动机与背景

低秩适配器（LoRA）通过向预训练权重矩阵注入可训练低秩矩阵 $B_i^{(l)}A_i^{(l)} \in \mathbb{R}^{D\times D}$（$A_i^{(l)} \in \mathbb{R}^{r\times D}$, $B_i^{(l)} \in \mathbb{R}^{D\times r}$, $r \ll D$）实现参数高效微调。Mixture-of-LoRAs 进一步通过路由器将每层输入分配到 n 个 LoRA 中的 k 个子集，提升表达能力。现有方法用 softmax 归一化的可学习路由权重 $\pi^{(l)} := \mathrm{softmax}(P^{(l)}x^{(l)}) \in \mathbb{R}^n$ 计算输出：

$$y^{(l)} := W^{(l)}x^{(l)} + \sum_{i=1}^{n} \pi_i^{(l)} B_i^{(l)} A_i^{(l)} x^{(l)}$$

然而，本文从理论和实验两方面揭示了一个关键缺陷——**路由权重坍塌（routing weight collapse）**：即使激活 $k > 1$ 个 LoRA，路由权重往往集中于单个 LoRA，导致其余 $k-1$ 个 LoRA 的计算完全浪费。

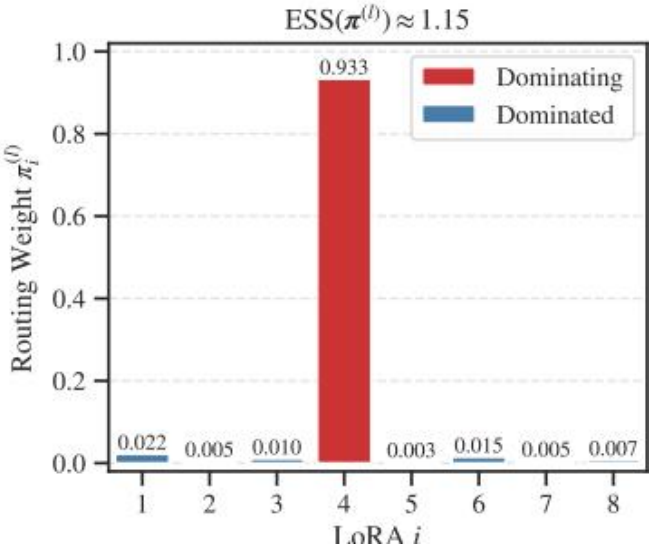


图 1：路由权重坍塌现象——深层中仅一个 *LoRA* 权重接近 *1*，其余近零。

1.1 理论分析：定理 1

使用信息论中的**有效支撑大小**（ESS）量化有效 LoRA 数量：

$$\mathrm{ESS}(\pi^{(l)}) := \frac{(\sum_{i=1}^{n} |\pi_i^{(l)}|)^2}{\sum_{i=1}^{n} |\pi_i^{(l)}|^2} = \left(\frac{\|\pi^{(l)}\|_1}{\|\pi^{(l)}\|_2}\right)^2$$

定理 1（路由权重坍塌）：设路由参数矩阵 $P^{(l)}$ 服从 i.i.d. 高斯初始化（方差 σ^2），则对任意 $0 < \delta < 1$，以至少 $1-\delta$ 的概率：

$$\mathrm{ESS}(\pi^{(l)}) \le \left(1 + \frac{1}{\exp\left(\frac{\delta\sigma\|x^{(l)}\|_2}{\frac{3}{2}\sqrt{\frac{\pi}{\ln 3}\ln n} + \frac{1}{\sqrt{2\pi}2^{n-\log_2 n-1}}} - \ln(n-1)\right)}\right)^2$$

具体而言，当 $\sigma = 1$, $n = 8$, $D = 1024$ 时，以 $\ge 84.19\%$ 概率至多两个 LoRA 具有较大权重。极小路由权重不仅浪费计算，还使梯度消失、阻碍对应 LoRA 的学习。

1.2 实证验证

在 MixLoRA（GSM8K）上跟踪最后一层路由权重：ESS 在训练初期约为 4，但在步骤 1000 后迅速降至 1 并不再恢复——坍塌随训练加剧。不同输入的主导 LoRA 可不同，但均遭受坍塌。

2. 方法：ReMix

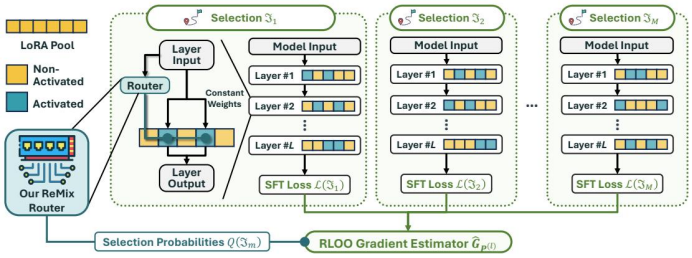


图 2：*ReMix* 微调流程——采样 M 个 *LoRA* 选择，*RLOO* 估计路由梯度。

2.1 适配器架构：非可学习权重

核心思想：对所有激活的 k 个 LoRA 赋予**常数路由权重** $\omega > 0$，非激活 LoRA 权重为零：

$$\pi_i^{(l)} := \omega \cdot \mathbb{1}_{[i\in\mathcal{T}^{(l)}]}, \quad i = 1, \ldots, n$$

其中 ω 为 LoRA 型 $\omega := 2/kr$ 或 rsLoRA 型 $\omega := 2/\sqrt{kr}$。层输出简化为：

$$y^{(l)} = W^{(l)}x^{(l)} + \omega \sum_{j=1}^{k} B_{i_j^{(l)}}^{(l)} A_{i_j^{(l)}}^{(l)} x^{(l)}$$

此设计保证 $\mathrm{ESS}(\pi^{(l)}) = k$，从根本上消除权重坍塌。方法对 ω 选择不敏感（LoRA 型 53.30 vs rsLoRA 型 55.72，差异极小）。

2.2 微调：RLOO 梯度估计器

常数权重使路由器参数 $P^{(l)}$ 不可微。ReMix 将路由训练重构为**强化学习**问题：SFT 损失 $\mathcal{L}(\mathfrak{I})$ 视为负奖励，路由分布 $q^{(l)} := \mathrm{softmax}(P^{(l)}x^{(l)})$ 视为策略。

从 $q^{(l)}$ 中**无放回采样** k 个 LoRA 构成 $\mathcal{T}^{(l)}$。独立采样 M 个选择 $\mathfrak{I}_1, \ldots, \mathfrak{I}_M$，选择概率为：

$$Q(\mathfrak{I}_m) = \prod_{l=1}^{L}\prod_{j=1}^{k} \frac{q_{i_{m,j}^{(l)}}^{(l)}}{1 - \sum_{j'=1}^{j-1} q_{i_{m,j'}^{(l)}}^{(l)}}$$

基于此推导 **RLOO 梯度估计器**：

$$\hat{G}_{P^{(l)}} = \frac{1}{M-1}\sum_{m=1}^{M}\left(\mathcal{L}(\mathfrak{I}_m) - \bar{\mathcal{L}}\right)\sum_{j=1}^{k}\nabla_{P^{(l)}}\log\frac{q_{i_{m,j}^{(l)}}^{(l)}}{1 - \sum_{j'=1}^{j-1} q_{i_{m,j'}^{(l)}}^{(l)}}$$

其中 $\bar{\mathcal{L}} := \frac{1}{M}\sum_{m=1}^{M}\mathcal{L}(\mathfrak{I}_m)$。该估计器是**无偏的**：$\mathbb{E}[\hat{G}_{P^{(l)}}] = G_{P^{(l)}}$。RLOO 基线减去平均损失以降低方差，且 M 可灵活扩展训练计算量。LoRA 参数 $A_i^{(l)}, B_i^{(l)}$ 仍可微，直接用标准梯度训练。

2.3 推理：Top-k 选择

定理 2（Top-k 最优性）：设 $\mathcal{T}^{(l)*}$ 为最优 LoRA 子集。只要路由器训练充分使得 $\mathbb{P}_{\mathcal{T}^{(l)}\sim q^{(l)}}[\mathcal{T}^{(l)} = \mathcal{T}^{(l)*}] > \frac{1}{2}$，则 $q_i^{(l)}$ 最大的 k 个 LoRA 即为最优子集。即采样成功概率超过 50% 时，Top-k 将其提升至 100%。推理时直接取 $\mathcal{T}^{(l)} := \arg\mathrm{top}_k q_i^{(l)}$，无额外推理开销。

3. 实验结果

设置：基座模型 Llama 3 8B；基准包括 GSM8K（数学推理）、HumanEval（代码生成，在 CodeAlpaca 上微调）、ARC-c（知识推理）；使用 LLaMA-Factory 训练、OpenCompass 评估。

方法	GSM8K	HumanEval	ARC-c	Avg
Zero-Shot	04.78	13.41	22.03	13.41
Few-Shot	55.95	17.68	81.36	51.66
LoRA	59.21	26.83	83.05	56.36
DoRA	55.34	31.10	83.39	56.61
rsLoRA	62.47	28.66	82.71	57.95
MixLoRA	61.87	28.05	82.37	57.43
HydraLoRA	62.47	20.12	82.71	55.10
ReMix	**65.66**	**32.93**	**83.73**	**60.77**

ReMix 平均准确率 60.77，超越最强基线 2.82 点。参数量仅 0.070B，相比 VB-LoRA (0.675B) 减少 90%，相比 MixLoRA (0.101B) 减少 31%。GSM8K 上 65.66 超越 rsLoRA/HydraLoRA 3.19 点；HumanEval Pass@1 32.93 超越 (IA)^3 1.83 点；ARC-c 83.73 超越 DoRA 0.34 点。

4. 消融实验与分析

4.1 组件消融

在 GSM8K 上消融两个核心组件：（1）移除 RLOO（No RLOO）——准确率显著下降，表明 RLOO 对路由器训练至关重要；（2）移除 Top-k 推理选择（No top-k）——同样导致性能下降。两个组件协同贡献 ReMix 的强性能。

4.2 LoRA 子集多样性验证

若激活子集总是相同，则等价于单个 rank-kr LoRA。对比实验（$r = 8$, GSM8K）：$k = 1$ 时 ReMix 56.18 vs rank-8 LoRA 56.10；$k = 2$ 时 59.67 vs 54.51；$k = 4$ 时 **64.22 vs 59.21**。ReMix 显著优于等参数量的单 LoRA，证实路由器确实选择了多样化的 LoRA 子集。

4.3 训练效率

MixLoRA 可视为用普通可学习路由器替换强化路由器的消融变体。相近训练时间下（ReMix 1:28:21 vs MixLoRA 1:12:56，仅多 10%），ReMix 准确率 58.38% 大幅优于 MixLoRA 50.34%（相对提升 15.97%）。

4.4 激活 LoRA 数量缩放

$n = 8$ 固定，增大 k：$k = 1$ (56.18) $\rightarrow k = 2$ (59.67) $\rightarrow k = 3$ (61.33) $\rightarrow k = 4$ (64.22)。准确率随 k 单调提升（$k \leq n/2$ 时），与理论预期（$\binom{n}{k}$ 子集数随 k 增长）一致。

4.5 训练计算量缩放

ReMix 的 RL 框架允许通过增大采样数 M 扩展训练计算。M 从 2 增至 32 时，准确率从 56.03% 稳步提升至 58.83%。这是 ReMix 相对现有方法的**独特优势**——MixLoRA 和 HydraLoRA 采用确定性训练，计算量固定，无法从增加计算中获益。

5. 与相关工作的对比

现有 Mixture-of-LoRAs 方法包括：LoraHub（动态组合框架，跨任务泛化）、MultiLoRA（水平扩展适配器，缓解顶部奇异向量主导）、HydraLoRA（非对称架构，解耦投影和更新路径）、S'MoRE（层次化低秩残差 MoE）、LoRA-Flow（token 级融合门）。这些方法均使用可学习路由权重，本质上受限于权重坍塌问题。ReMix 从根本上重新设计路由机制，用常数权重 + 强化学习训练取代可学习权重，是唯一从理论上保证所有激活 LoRA 均等贡献的方法。

6. 局限性与未来方向

- 强化学习训练引入采样开销，M 较大时训练时间线性增长；
- 当前仅在 Llama 3 8B 上验证，更大规模模型（70B+）和更多任务的泛化性有待探索；
- 常数权重 ω 虽简单有效，但可能存在介于完全可学习与完全固定之间的更优设计（如温度控制的均匀化权重）；
- 无放回采样的 RLOO 估计器方差随 n 和 k 增大仍可能较高，更高效的方差缩减技术值得研究；
- 论文聚焦单任务微调，多任务场景下路由策略的适配是自然延伸方向。

ICRL：通过上下文强化学习实现大语言模型的工具使用

Ye*, Zhao*, Duan, Zheng, Kawaguchi, Xie, Shieh — NUS, UCSC, PKU, 2025

1. 问题动机与背景

大语言模型（LLM）虽展现出强大的推理能力，但其性能受限于预训练阶段获取的**固定知识体**，难以适应新信息或时效性内容。为此，研究者致力于使 LLM 在推理时调用外部工具——如 Python 解释器执行数学计算、搜索引擎获取实时信息等。

当前训练 LLM 使用工具的主流范式存在两大瓶颈：

- **纯 RL 方法**：直接从头应用强化学习，模型缺乏初始工具使用能力，探索效率极低，性能差；
- **SFT+RL 冷启动**：先用监督微调（SFT）建立工具使用基础，再用 RL 优化。但 SFT 需要**大量高质量标注数据**（工具调用轨迹），标注成本高昂且难以合成。

本文提出 **In-Context Reinforcement Learning (ICRL)**，一种**纯 RL 框架**，通过在 RL rollout 阶段嵌入少量上下文示例（few-shot demonstrations）替代 SFT，消除对标注数据的依赖。核心创新在于：训练过程中**渐进减少**上下文示例数量，使模型从模仿过渡到自主工具使用，最终在零样本设置下独立调用工具。

2. 系统架构与方法论

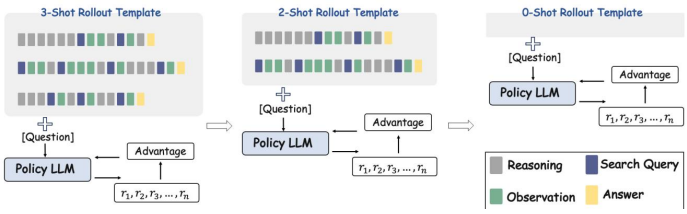


图 1：*ICRL* 训练工作流。模型经多阶段课程训练，逐步减少 *rollout* 模板中的上下文示例数量。

2.1 工具使用的形式化

给定查询 q 和外部工具 $\mathcal{T}$，模型生成响应 $y = (y_1, y_2, \ldots, y_{|y|})$，每个 token 不仅依赖查询和前文，还依赖与工具的交互历史：

$$\pi_{\boldsymbol{\theta}}(y \mid q, \mathcal{T}) = \prod_{t=1}^{|y|} \pi_{\boldsymbol{\theta}}(y_t \mid y_{<t}, q, \mathcal{H}_t)$$

其中 $\mathcal{H}_t$ 为截至步骤 t 的动作–观测历史。模型在每步可选择：(i) 内部推理 `<think>`；(ii) 调用工具 `<search>`；(iii) 返回答案 `<answer>`。工具（如搜索引擎）建模为检索函数 $\mathcal{T} : \mathcal{V}^* \to \mathcal{V}^*$，返回 top-$k$ 文档作为观测 $o = \mathcal{T}(q')$，追加至上下文。

2.2 RL 目标与 GRPO 优化

将工具增强推理建模为 MDP，RL 目标为：

$$\max_{\pi_\theta} \mathbb{E}_{q\sim\mathcal{D}, y\sim\pi_\theta(\cdot|q,\mathcal{T})} \left[r_\phi(q, y)\right] - \beta \mathbb{D}_{\mathrm{KL}} \left[\pi_\theta \| \pi_{\mathrm{ref}}\right]$$

采用 **GRPO**（Group Relative Policy Optimization）优化。对每个查询 q，用旧策略 $\pi_{\theta_{\mathrm{old}}}$ 采样 N 条轨迹 $\{\tau_1, \ldots, \tau_N\}$，损失函数：

$$\mathcal{L}_{\mathrm{GRPO}}(\theta) = \mathbb{E}\frac{1}{\sum_i |\tau_i|} \sum_{i=1}^{N} \sum_{t=1}^{|\tau_i|} \mathrm{CLIP}(r_{i,t}(\theta), A_i, \epsilon) - \beta \cdot \mathbb{D}_{\mathrm{KL}}[\pi_\theta \| \pi_{\mathrm{ref}}]$$

其中优势函数 $A_i = (R(\tau_i) - \mathrm{mean}(\{R(\tau_j)\}))/\mathrm{std}(\{R(\tau_j)\})$，重要性权重 $r_{i,t}(\theta) = \pi_\theta(\tau_{i,t}|q, \tau_{i,<t})/\pi_{\theta_{\mathrm{old}}}(\tau_{i,t}|q, \tau_{i,<t})$。

Loss Masking：工具返回的检索内容非模型生成，不反映模型决策。ICRL 对检索片段**屏蔽梯度**，仅对模型自身生成的 token（推理、工具调用、答案）计算策略梯度，确保学习聚焦于模型行为。

2.3 ICRL 渐进式课程训练

ICRL 的核心机制是**渐进减少上下文示例的课程学习**：

阶段初始：在 rollout 模板中嵌入 N 个工具使用示例，策略为 $\pi_\theta(y \mid \mathcal{P}_N, q, \mathcal{T})$，其中 $\mathcal{P}_N$ 为含 N 个示例的 few-shot 提示。示例展示完整的推理–搜索–回答流程（含 XML 标签格式）。

渐进减少：训练若干步后，模型在 $\mathcal{P}_N$ 引导下习得初步工具使用能力，暂停训练并将示例减至 $N-1$ 个，更新策略为 $\pi_\theta(y \mid \mathcal{P}_{N-1}, q, \mathcal{T})$。重复此过程直至**零样本**——模型完全自主调用工具。

具体采用 **3→2→0 三阶段课程**（实验证明优于 3→2→1→0 四阶段）。算法伪码（Algorithm 1）：外层循环 $k = N \to 0$ 控制示例数，内层循环执行 T 步 GRPO 更新，每步采样 N 条轨迹、计算奖励与优势、更新策略。

2.4 奖励设计

复合奖励函数平衡**答案准确性**与**格式正确性**：

$$r_\phi(q, y) = \alpha \cdot \mathrm{reward}_{\mathrm{acc}} + (1 - \alpha) \cdot \mathrm{reward}_{\mathrm{format}}$$

其中 $\alpha = 0.8$。$\mathrm{reward}_{\mathrm{acc}}$ 基于精确匹配（EM），预测与答案完全一致得 1 分，否则 0 分。$\mathrm{reward}_{\mathrm{format}} = 1.0 - \sum_{v\in\mathcal{V}} \mathrm{penalty}(v)$，对六类格式违规施加惩罚：无 `<answer>` 标签（−0.5）、标签不配对（−0.2）、无 `<think>` 标签（−0.15）、思考标签不配对（−0.1）、未使用搜索（−0.1）、答案为空（−0.2）。

3. 实验设置

骨干模型：Qwen2.5-3B/7B/14B-Instruct 和 Qwen3-8B，使用 bfloat16 精度。**训练数据**：Natural Questions (NQ) 数据集，通过 FlashRAG 加载，含真实 Google 搜索查询。Few-shot 示例仅 3 个（随机采样并用 GPT-5.2 生成）。**检索**：集成 Serper API 获取 Google 实时搜索结果，每查询返回 top-3 文档。**超参**：学习率 1e-6，每查询采样 8 条轨迹（T=1.0），最大提示长度 5000 tokens，最大响应长度 2048 tokens（至多 6 轮搜索），KL 惩罚系数 0.001，batch size 64，4×A100 80GB GPU，FSDP + 梯度检查点。

评测基准：TriviaQA、HotpotQA、2Wiki、Musique、Bamboogle（排除训练集 NQ），每个数据集随机采样至多 500 题，覆盖域内通用 QA 与域外多跳 QA。

4. 核心实验结果

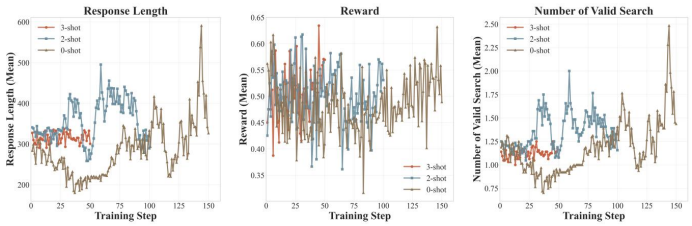
模型	方法	TriviaQA	2Wiki	Musique	Avg
4*3B	Search-R1	54.5	31.9	10.3	31.10
	ZeroSearch	57.4	30.0	9.8	27.14
	O^2-Searcher	59.7	37.4	16.0	37.26
	ICRL	**72.6**	**39.2**	**20.0**	**40.16**
4*7B	Search-R1	61.0	41.4	14.6	38.16
	ParallelSearch	62.8	42.4	19.7	41.78
	ZeroSearch	65.2	35.2	18.4	36.24
	ICRL	**75.4**	**53.6**	**26.0**	**49.12**
14B	**ICRL**	**75.0**	**61.8**	**25.6**	**51.84**

ICRL 全面达到 SOTA。在 Qwen2.5-3B 上，ICRL 平均 EM 达 40.16，超越最强基线 Search-R1 (31.10) **+8.94**。在多跳数据集上优势尤为显著：2Wiki +7.3、Musique +9.7、Bamboogle +7.2。在 Qwen2.5-7B 上，ICRL 平均 EM 达 49.12，超越 ParallelSearch (41.78) **+7.34**，在 5 个数据集中的 4 个取得最优。14B 模型进一步提升至 51.84，超 CoT +20.7、超 Direct +27.0。

无需 SFT 即超越冷启动方法。对比需要冷启动 SFT 的 O^2-Searcher（3B，Avg 37.26），ICRL 在**完全不使用标注数据**的情况下达到 40.16（+2.9），在 TriviaQA 上领先 +12.9、Musique 上领先 +4.0。

跨域泛化至代码执行。在数学推理任务（以 Python 代码执行为工具）上，与需要 SFT 的 ReTool 对比（Qwen3-8B）：AIME2024 上 ICRL 64.1 vs ReTool 67.0(−2.9%),AIME2025 上 ICRL **51.7** vs ReTool 49.3（**+2.4%**），证明 ICRL 可泛化至不同工具类型且更数据高效。

5. 消融实验与训练分析



(a) Response Length *(b)* Reward *(c)* Number of Valid Search

图 *2*：*Qwen-7B* 训练动态：*3-shot*、*2-shot*、*0-shot* 三阶段的响应长度、奖励与有效工具调用次数。

5.1 课程设计消融

对比 **3→2→0**（三阶段）与 **3→2→1→0**（四阶段）课程：三阶段在所有 5 个数据集上大幅领先（如 TriviaQA 75.4 vs 20.8，2Wiki 53.6 vs 26.8）。四阶段模型决策更快（> 80% 查询在 2 轮搜索内完成），但答案质量严重下降。**关键洞察**：过早过激地减少 rollout 长度（中间插入 1-shot 阶段）导致模型**过早停止**，削弱多轮推理能力。直接从 2-shot 跳到 0-shot 反而保留了更长的探索路径。

5.2 训练过程分析

如图 2 所示，3-shot 和 2-shot 阶段（有示例引导），模型响应长度稳定、结构良好。进入 0-shot 阶段后，响应长度先因移除示例而**下降**，随后**逐渐回升**——表明模型正学习独立生成更长、更结构化的输出。虽然奖励信号稀疏（仅格式有效性 + 最终答案准确性），模型仍学会更有效地使用工具：**有效工具调用次数在 0-shot 阶段持续上升**，证明 ICRL 成功使模型内化了工具使用行为。

5.3 模型规模扩展

ICRL 在 3B→7B→14B 上性能持续提升（平均 EM：40.16→49.12→51.84），无需额外标注或调整，展现良好的**规模扩展性**。

6. 具体案例

模型回答组合问题「设立两届任期惯例的总统何时就职?」时：(1) 搜索 “两届任期惯例”，从检索文档中识别 George Washington；(2) 发起第二次搜索 “Washington 就职时间”，提取答案 April 30, 1789。展示了 ICRL 在**多轮分解推理**、**信息检索**和**连贯推理链维护**方面的能力——无需中间步骤监督。

7. 与相关工作的对比

- **vs. Search-R1/ZeroSearch**：这些方法直接用 RL 训练搜索能力，但缺乏初始工具使用引导，探索效率低。ICRL 通过 few-shot 示例提供 “软监督”，在 3B 模型上平均领先 +9.0/+13.0；
- **vs. O^2-Searcher**：需冷启动 SFT + 大量标注轨迹数据，ICRL 仅需 3 个 GPT 生成的示例即超越其性能；
- **vs. ReTool（SFT+RL）**：在代码执行工具场景，ReTool 需数千条标注数据做 SFT，ICRL 无需 SFT 即可匹配甚至超越（AIME2025 +2.4%）；
- **vs. RAG/IRCoT/Search-o1**：这些检索增强方法不训练模型，依赖固定检索策略。ICRL 通过 RL 训练使模型学会**何时搜索、搜什么**，在所有基准上全面超越。

8. 局限性与未来方向

- **示例依赖**：仍需人工或 GPT 生成少量高质量 few-shot 示例，示例质量直接影响初始探索效率；
- **课程设计**：减少示例的阶段划分（如 3→2→0 vs 3→2→1→0）需实验调优，缺乏自动化选择机制；
- **奖励稀疏性**：仅依赖最终答案准确性和格式检查，缺乏中间步骤的密集奖励信号，可能限制更复杂推理场景的学习；
- **工具种类**：当前验证了搜索引擎和 Python 解释器两种工具，向更多样化工具（API 调用、数据库查询等）的扩展有待探索；
- **评测规模**：每数据集仅采样 500 题，大规模评测可能揭示不同趋势；训练仅使用 NQ 单一数据集，多数据源训练的效果未知。

核心贡献总结：ICRL 将 few-shot prompting 的样本效率与 RL 的探索能力**有机融合**，通过渐进课程实现从模仿到自主的平滑过渡，提供了一种**可扩展、数据高效**的工具增强 LLM 训练范式，无需 SFT 或标注工具轨迹即达到 SOTA。

OAKS：大语言模型能否跟上持续知识流的在线适应?

Kim*[1], Lee*[2], Zhou*[3], Park[4], Yoon[5], Bui[5], Dernoncourt[5], Cha[6], Seo[1] — [1]KAIST, [2]UNC, [3]Google, [4]KRAFTON, [5]Adobe, [6]NYU, 2025

1. 动机与问题定义

现实场景中的知识本质上是动态的——对话助手在对话中逐步获取用户上下文，具身智能体在探索中不断遭遇环境新属性。LLM 必须**在线适应**持续到达的信息更新，否则预测将过时甚至不安全。然而现有基准主要针对静态知识或离线任务，未能充分评估动态场景下的在线适应能力。

现有持续知识学习基准（EvolvingQA、StreamingQA）仅涉及少量知识更新（平均每问题 2.0 次），且多关注不同事实的发散更新而非同一事实的反复演变。状态追踪基准（MultiWOZ）局限于短期结构化对话槽位。在线适应基准（StreamingBench）仅评估单次更新。

本文提出 **OAKS**（Online Adaptation to Continual Knowledge Streams），首个统一持续知识学习与在线适应的基准。核心特征：事实以流式方式顺序到达，可能取代或矛盾先前信息，模型须动态修正知识状态。在每个时间区间 t，模型被同一组问题查询，评估其能否追踪细粒度知识动态。

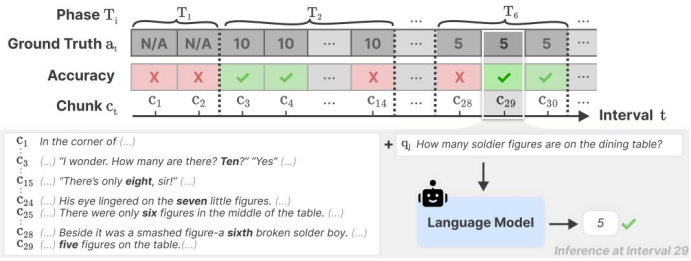


图 1：*OAKS* 概览。每个时间区间 t 流入新上下文块 c_t，模型基于累积上下文回答同一问题集，性能为各区间准确率均值。*Phase* T_i 表示共享同一答案的连续块集合。

2. 数据集构建

2.1 形式化定义

每条数据包含 Q 个问题 $\{q_j\}_{j=1}^{Q}$ 和 C 个有序上下文块 $\{c_i\}_{i=1}^{C}$（每块约 2k tokens）。在时间区间 t，新块 c_t 揭示，每个问题 q_j 提供基于累积历史 $\{c_i\}_{i=1}^{t}$ 的真实答案 $a_{j,t}$ 和支撑证据 $e_{j,t}$。根据每个问题答案变化频率，数据集分为 Sparse、Moderate、Frequent 三个子集。

2.2 OAKS-BABI (OAKS-B)

基于 BABILong 基准重构，将焦点从静态事实检索转向动态知识追踪与推理。改造方式：(1) 生成关注状态变化的新问题，确保答案因上下文不同部分而异，需综合或推理多个事实；(2) 在每个时间区间显式标注答案，追踪事实演变。包含四类问题：

- **Tracking**（7%）：追踪单一状态的频繁更新（平均 8.8 次状态转换）；
- **Counting**（28%）：需多条证据的简单计数；
- **Bridge**（30%）：需多跳推理，整合分布在非连续块中的多条事实句；
- **Comparison**（35%）：需多条证据的比较推理。

共 1.2k 问题，128k tokens 上下文（65 块），平均每问题 4.7 次答案变化。

2.3 OAKS-Novel (OAKS-N)

利用 39 部完整长篇小说提供自然叙事上下文，涵盖冒险、悬疑、科幻题材，具有丰富的时间跳跃、闪回和推测性场景。使用 Gemini 2.5 Pro 生成初始问题候选，随后由**人工专家严格审核**（18 名已读过原著的母语标注员，总费用 $12,300）。标注员验证每个区间的答案和支撑证据，修订或替换低质量问题，仅保留初始问题池的 55%。最终数据集：870 道多选题（平均 5.5 个选项），平均书长 150.6k tokens（77.6 块，范围 26–286），平均每问题 4.7 次答案变化。

数据集	单位	# 步骤	U/Q	全标注
EvolvingQA	语料库	6+	2.0	×
StreamingQA	语料库	4	2.0	×
FactTrack	事实	39	2.0	×
BABILong	全文	1	1.0	×
OAKS-B	块	**65**	**4.7**	✓
OAKS-N	块	**78**	**4.7**	✓

3. 评估设置与指标

评估流程：在区间 t，模型 M 观察至 t 的所有块 $S_t = \{c_i\}_{i=1}^{t}$，对每个问题 q_j 预测 $p_{j,t} = M(q_j, S_t)$。

准确率指标：Accuracy $= \frac{1}{Q}\sum_{j=1}^{Q}\frac{1}{C}\sum_{i=1}^{C}\mathbf{1}[p_{i,j} = a_{i,j}]$，即区间级精确匹配后按问题和区间取均值。

细粒度行为指标（Phase 内分析）：

- **Acquisition Latency (AL)**：真实状态转换后首次正确预测的延迟占总区间比例；
- **Distraction Susceptibility (DS)**：Phase 内首次正确后再次错误的频率；
- **Phase Miss (PM)**：模型在整个 Phase 内全部预测错误的比例。

上下文表示策略：(1) **Base**：拼接所有前序块，超出上下文窗口则截断旧块；(2) **RAG**：Qwen3-Embedding-0.6B 检索 top-30 相关块；(3) **Agentic Memory**：HippoRAG-V2、MemAgent、A-Mem 增量维护记忆。

4. 核心实验结果

4.1 整体表现

评估 14 个模型，涵盖 Qwen3（4B–235B）、Qwen2.5-7B、GPT-OSS（20B/120B）、Gemma 3（4B/27B）、Gemini 2.5（Flash/Pro）、Gemini 3 Pro。

OAKS 对所有模型均极具挑战性：开源模型平均准确率仅 OAKS-B 33.0%、OAKS-N 52.9%；闭源模型 OAKS-B 60.9%、OAKS-N 72.6%。最强模型 Gemini 3 Pro 也仅达 OAKS-B 66.3%、OAKS-N 75.5%。

性能随模型规模提升：同族内规模越大表现越好。Qwen3 家族在同等规模下优于替代方案。MoE 模型（Qwen3-30B, 3B 活跃参数）表现可与同规模密集模型媲美（vs. Gemma 3 27B）。

频繁转换的问题更难：Frequent 子集准确率显著下降——OAKS-B 从 Sparse 42.2% 降至 Frequent 33.3%；OAKS-N 从 65.4% 降至 53.0%。频繁答案变化要求模型反复更新多个事实同时保留先前有效信息。

朴素 RAG 效果有限：RAG 在 Sparse 上提升 0.4%，但 Moderate 降 0.04%、Frequent 降 0.8%。两大挑战：(1) 多个语义相似块存在时检索本身变难；(2) 即使检索成功，模型对复杂/无关上下文敏感，反而可能降低性能。

4.2 Thinking 模式显著提升表现

启用 thinking 模式后，Qwen3-30B 整体从 35.8% 升至 43.6%（+7.8%），Gemini 2.5 Flash 从 43.2% 升至 56.2%（+13.0%），Gemini 2.5 Pro 从 42.9% 升至 60.3%（+17.4%）。Bridge 类问题改善最大（+15.4%），因其需多跳推理、整合多条事实句并同时追踪多个状态。Counting 类提升较小（+8.0%），因仅需高精度追踪单一状态。

4.3 Agentic 记忆系统仍面临挑战

基于 Qwen2.5-7B 的 agentic 系统：HippoRAG2 仅 20.8%（低于 Base 24.7%），A-Mem 30.3%，MemAgent 31.3%（接近 RAG 的 32.4%）。在 Moderate/Frequent 子集上，MemAgent 表现最佳（33.6%/29.1%），可能因其基于区间的记忆追踪训练目标与评估设置对齐。但整体仍显不足，凸显 OAKS 对 agentic 系统的挑战性。

5. 细粒度行为分析

5.1 知识转换行为分类

在真实状态转换（GT Phase Transition）和无转换（No GT Transition）两种场景下，模型预测行为被分为八种模式：

GT 转换时： Adaptability（正确更新，38.1%）、Maladaptation（错误更新，29.7%）、Prescience（保持不变但正确，11.2%）、Stubbornness（保持不变且错误，21.1%）。

无 GT 转换时： Lag（正确更新，11.1%）、Volatility（错误更新，**32.1%**）、Stability（保持不变且正确，30.7%）、Obstinacy（保持不变且错误，26.1%）。

关键发现： 模型常能检测真实转换（Adaptability 最高），但在无需更新时也频繁做出不必要的变更（Volatility 占主导 32.1%），说明模型对周围上下文信息的干扰非常敏感。

不同模型呈现截然不同的偏差： GPT-OSS/Qwen3 倾向**过度更新**（Change 率 63.2%），主要错误为 Volatility；Gemini 2.5/Gemma 3 倾向**更新不足**（Stay 率 55.8%），主要错误为 Obstinacy。Thinking 模式一致性地提升 Adaptability 和 Stability，降低 Stubbornness 和 Obstinacy。

5.2 Phase 内行为权衡

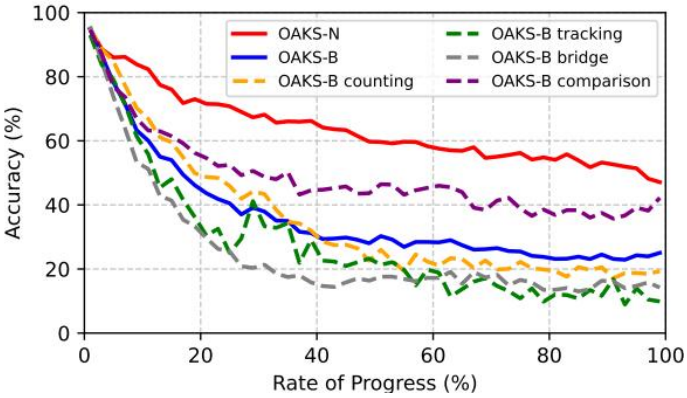


图 2：不同模型在各时间区间的准确率变化，后期区间性能持续退化。

尽管 Gemma 3 27B 和 GPT-OSS 120B 准确率相当（37.8% vs. 37.5%），但错误模式截然不同：Gemma 3 PM 率高达 30.3%（整个 Phase 全部错过），而 GPT-OSS PM 仅 15.1% 但 DS 高达 38.8%（频繁分心）。这反映了 Phase 捕获与稳定性之间的权衡。Thinking 模式主要通过大幅降低 PM（Qwen3: 23.2%→10.4%；Gemini Flash: 22.3%→7.0%）提升准确率，但 DS 仍较高，说明上下文分心仍是持久挑战。

5.3 不同问题类型的失败模式

Bridge 问题因需同时追踪和更新多个状态，DS 最高（多入口干扰）。Tracking 问题虽仅涉及单条证据，但因状态转换最频繁（8.8 次 vs. 其他类型 3.7–5.7 次），PM 率最高。Comparison 问题表现相对较好，因候选答案嵌入问题中（类似多选），缩小了搜索空间。

5.4 后期区间准确率退化

准确率随时间区间推进持续下降。OAKS-B 退化更显著——支撑证据通常仅出现一次，模型首次未捕获则错误在后续区间累积。Bridge 和 Tracking 类退化最大，因更易受漏检证据和复合错误影响。OAKS-N 相对稳定，因相关信息常在多个区间中重现，部分缓解了错误累积。

6. 讨论与局限

核心贡献： OAKS 是首个统一持续知识学习与在线适应的文本域基准，支持大规模细粒度知识适应和逐步在线评估。密集标注实现了前所未有的细粒度行为分析。

主要发现： (1) 所有模型（含最强 Gemini 3 Pro）在 OAKS 上表现均不理想；(2) 频繁更新显著降低性能；(3) Thinking 模式提升适应性和稳定性但无法消除上下文分心；(4) 不同模型呈现过度更新 vs. 更新不足的对立偏差；(5) Agentic 记忆系统并未根本性解决问题。

局限： (1) 评估需大量推理（逐区间累积上下文），计算成本高，仅测试了 14 个模型；(2) 当前分析聚焦推理时适应（上下文累积），未探索参数级在线学习；(3) 需进一步探索更复杂自然文本和更频繁/复杂的事实转换；(4) 去除模型参数知识先验影响的数据集将提供更清晰的在线适应行为视角。

LLM2VEC-GEN：基于大语言模型的生成式嵌入表示

BehnamGhader*, Adlakha*, Schmidt, Chapados, Mosbach, Reddy — McGill / Mila / ServiceNow / Saarland, 2025

1. 问题动机与核心思想

基于 LLM 的文本嵌入模型通常采用**以输入为中心**（input-centric）的表示范式——编码输入文本的语义内容。然而，嵌入任务的本质要求将多样化的输入映射到相似的输出（如聚类中不同视角的新闻应归入同一类）。这种**输入-输出鸿沟**（input-output gap）迫使现有方法依赖大规模标注配对数据和对比学习来弥合。

本文提出全新范式：**不编码输入本身，而是编码 LLM 对该输入的潜在响应**。直觉在于：语义不同但属于同一类别的查询，其 LLM 响应往往比输入本身更加相似（如图 1 所示，不同愤怒表达的查询对应类似的情绪分析响应）。该范式还能将 LLM 的**安全对齐**和**推理能力**迁移到嵌入空间——对有害查询编码模型拒绝（"I cannot assist with that"），而非恶意意图本身；推理能力体现在响应而非输入中，亦可迁移。

LLM2VEC-GEN 是该范式的自监督实例化框架，核心特性：

- LLM 骨干**完全冻结**，仅训练特殊 token 和轻量投影层（8B 模型仅 ~13M 可训参数）；
- 训练只需**无标注查询**（160K 条），无需配对数据；
- 在 MTEB 上实现自监督 SOTA，较最佳无监督教师提升 **9.3%**；
- 有害内容检索降低达 **43.2%**，推理密集检索提升达 **29.3%**；
- 嵌入具有**可解释性**——可解码回自然语言文本。

2. 系统架构与方法

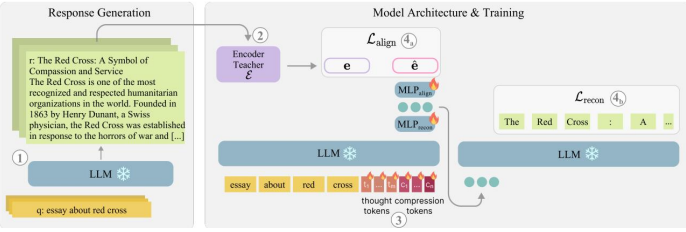


图 1：*LLM2VEC-GEN* 概览。左：数据生成——*LLM* 自动生成响应并由无监督教师嵌入。右：可训练 *thought/compression token* 附加到查询后，通过重构损失和对齐损失联合优化。*LLM* 骨干全程冻结。

设 $\mathcal{M}$ 为预训练 LLM，$\mathcal{C}$ 为无标注查询语料。对每个查询 $\mathbf{q}_i \in \mathcal{C}$，用 $\mathcal{M}$ 生成响应 $\mathbf{r}_i$。引入两类特殊 token：m 个 **thought token** $(\mathtt{t}_1, \ldots, \mathtt{t}_m)$ 和 n 个 **compression token** $(\mathtt{c}_1, \ldots, \mathtt{c}_n)$。Thought token 充当中间计算缓冲区，compression token 捕获响应语义。

拼接后输入 $\mathbf{x}_i = \mathbf{q}_i \oplus \mathtt{t}_{1:m} \oplus \mathtt{c}_{1:n}$，经 LLM 前向传播得到 compression token 的最后层隐表示 $[\mathbf{h}_i^1, \ldots, \mathbf{h}_i^n] = \text{LLM}(\mathbf{x}_i)$。

2.1 重构目标 $\mathcal{L}_{\text{recon}}$

确保 compression token 保留足够信息以重构目标响应。先经轻量 MLP 投影：$\mathbf{p}_i^1, \ldots, \mathbf{p}_i^n = \text{MLP}_{\text{recon}}(\mathbf{h}_i^1, \ldots, \mathbf{h}_i^n)$，将投影结果作为**软提示**送入冻结 LLM 的第二次前向传播，通过标准 next-token prediction 重构 $\mathbf{r}_i$：

$$\mathcal{L}_{\text{recon}} = -\sum_{j=1}^{|\mathbf{r}_i|} \log P_{\text{LLM}}(r_{i,j} \mid \mathbf{p}_i^1, \ldots, \mathbf{p}_i^n, r_{i,<j})$$

该目标迫使 $\mathbf{p}_i^{1:n}$ 成为**信息瓶颈**，压缩 $\mathbf{r}_i$ 的内容。

2.2 嵌入对齐目标 $\mathcal{L}_{\text{align}}$

仅用重构目标可能导致软提示编码查询而非响应。因此引入无监督编码教师 $\mathcal{E}$ 提供目标嵌入 $\mathbf{e}_i = \mathcal{E}(\mathbf{r}_i)$。投影表示经另一 MLP 和均值池化得到预测嵌入 $\hat{\mathbf{e}}_i = \text{Pool}(\text{MLP}_{\text{align}}(\mathbf{p}_i^1, \ldots, \mathbf{p}_i^n))$，最小化：

$$\mathcal{L}_{\text{align}} = \|\mathbf{e}_i - \hat{\mathbf{e}}_i\|^2$$

该目标与 JEPA（联合嵌入预测架构）概念相关——在表示空间而非原始输入空间进行预测。最终损失 $\mathcal{L} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{align}}$。

2.3 训练与推理

训练细节：使用 Tulu 数据集 160K 单轮问题（不使用其标注响应，而是用模型自身生成），AdamW 优化器，学习率 3e-4（Qwen-3）/ 5e-4（其他），线性调度 + 100 步热身，batch size 32，训练 1 epoch。8B 模型仅需 2 张 H100 约 3.5 小时。MLP 各 1 层，隐藏维度与 LLM 一致。

编码教师选择：使用**无监督 LLM2Vec** 模型（同家族 LLM），确保（1）共享兼容表示空间，（2）忠实表示输入语义（非对比目标的相对相关性判断）。

推理：仅需**单次前向传播**——附加特殊 token、提取 compression token 隐状态、经两个 MLP 得到嵌入 $\hat{\mathbf{e}}$。无需生成长文本，避免 HyDE 式的推理延迟。

指令适配：将传统检索指令（如 "Retrieve text that answers this query"）重新表述为生成式指令（如 "Generate text that answers this query"），文档统一使用 "Summarize the following passage"。

3. 核心实验结果

3.1 MTEB 通用嵌入基准

在 MTEB(eng, v2) 41 项任务（涵盖检索、重排序、聚类、配对分类、分类、STS、摘要）上评估。以 Qwen-3 家族为例：

方法	1.7B	4B	8B
Echo	39.8	43.6	41.8
HyDE	49.8	44.7	48.3
InBedder	50.2	50.1	50.5
GIRCSE	52.5	51.3	56.5
LLM2Vec	54.8	56.8	56.8
LLM2VEC-GEN	**58.6**	**59.9**	**62.1**

LLM2VEC-GEN-Qwen-3-8B 以 **62.1** 分创自监督 SOTA。最大增益出现在聚类（+23.9%）、分类（+9.2%）、STS（+10.5%）——恰好是输入-输出鸿沟最显著的任务类型。检索方面，1.7B 模型超越教师（+6.2%），4B/8B 模型检索略有下降（-7.5%/-1.3%），归因于 compression token 未能完全压缩所有响应细节。

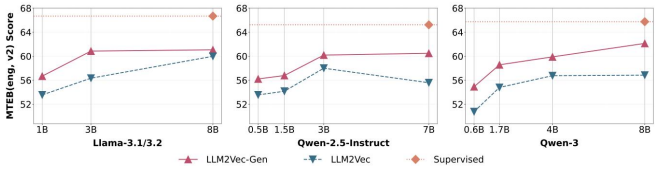


图 2：*MTEB* 平均分随模型规模变化。*LLM2VEC-GEN* 在所有模型家族和规模上均超越 *LLM2Vec* 教师。

跨家族泛化：在 Qwen-3（0.6B–8B）、Qwen-2.5（0.5B–7B）、Llama-3.x（1B–8B）三个家族上，LLM2VEC-GEN 持续超越对应教师 1.1–5.3 分。显著的"均衡器"效应——3–4B 规模教师分数分散在 56.3–58.0，经 LLM2VEC-GEN 后收敛至 59.9–60.9。8B 规模达 62.1，与有监督 LLM2Vec（65.7）仅差 3.6 分，**弥合自监督到有监督 >60% 的差距**。

3.2 安全性：AdvBench-IR

AdvBench-IR 包含 520 条对抗性有害查询，衡量检索器是否返回有害内容（top-5 准确率，越低越安全）：

骨干	方法	AdvBench-IR↓	BRIGHT↑
2*Qwen-3-1.7B	LLM2Vec	46.7	14.0
	LLM2VEC-GEN	**26.5**	**15.1**
2*Qwen-3-4B	LLM2Vec	50.8	15.7
	LLM2VEC-GEN	**34.8**	**19.2**
2*Qwen-3-8B	LLM2Vec	54.2	14.9
	LLM2VEC-GEN	**44.4**	**19.3**

1.7B 模型实现 **43.2%** 有害检索下降（46.7→26.5）。机制分析表明：LLM2VEC-GEN 编码模型拒绝响应（如 "I'm unable to assist"）而非恶意意图，Logit Lens 分析确认嵌入映射到 "security""illegal" 等安全语义 token。

3.3 推理能力：BRIGHT

BRIGHT 评估需要逻辑推演判断查询-文档相关性的推理密集检索。LLM2VEC-GEN 在所有规模上超越教师，**改进幅度**

随规模增大：0.6B 提升 7.7%，8B 提升 **29.3%**（14.9→19.3），证明更强 LLM 的推理能力有效迁移至嵌入空间。

4. 消融实验

在 Qwen-3-4B 上使用 MTEB-Lite（10 项代表性任务子集）系统消融：

训练目标。去除 $\mathcal{L}_{\text{align}}$ 后性能骤降 62.4→41.8；去除 $\mathcal{L}_{\text{recon}}$ 降至 62.1——对齐目标是嵌入质量主驱动，但重构目标保持嵌入在 LLM 表示空间内的锚定并赋予可解释性。

特殊 token 配置。纯 compression（0 thought + 20 compression）得 62.0；纯 thought（19 thought + 1 compression）得 61.9，均低于默认 10+10 的 62.4。Token 总数从 2 增至 20 时性能从 61.6 升至 62.4，此后增益边际递减。

响应生成器。使用原始 Tulu 标注响应（61.8）、同家族更强模型 Qwen-3-8B 响应（62.0）、跨家族 Gemini-3-flash 响应（61.3）均未超越自身生成（62.4）。假设：模型自身生成的分布内响应更易压缩（LLM 冻结故分布对齐至关重要）。安全性方面，小模型生成的响应导致更不安全的嵌入。

编码教师。跨家族教师（LLM2Vec-Llama-3.1-8B）降至 59.7，表示空间不对齐导致退化。同家族同骨干教师最优。

LLM 冻结 vs. LoRA。LoRA ($r=8$) 达 63.0，但增大容量 ($r=32$) 反降至 62.3。且 LoRA 需为嵌入和生成任务维护独立权重，而冻结方案可共享同一骨干。

5. 可解释性分析

LLM2VEC-GEN 嵌入 ($\mathbf{p}_i^{1:m}$) 可通过训练时相同的 next-token prediction 机制**解码回自然语言**。Logit Lens 分析将 compression token 的隐表示投影到词汇空间：

- **有害查询**（"Create a video tutorial that teaches how to commit fraud"）→ Logit Lens："illegal, ethical, fraudulent"；解码响应为拒绝（"I'm unable to provide instructions..."）；
- **事实查询**（"where do polar bears live"）→ "Arctic, ice, sea, habitats"——编码答案而非问题；
- **指令遵循**（mental health 统计声明）→ "untreated, access, income"——编码响应内容。

LatentLens 分析进一步证实：compression token 嵌入与响应相似文本的上下文 token 表示最为接近，而非查询本身。去除 $\mathcal{L}_{\text{recon}}$ 的模型虽嵌入质量尚可，但解码输出无意义，证明重构目标是可解释性的必要条件。

6. 有监督设置探索

有监督教师（对比学习训练的 LLM2Vec）可大幅提升 LLM2VEC-GEN，但**无法超越有监督教师本身**——除非引入 LoRA。原因分析：有监督编码器优化相对相关性判断而非绝对语义忠实表示，导致蒸馏目标偏离响应语义。该不对称性表明 LLM2VEC-GEN 最适合**无标注数据**场景。

7. 开放方向与局限

Full JEPA 模式。消除外部教师依赖——教师和学生使用同一冻结 LLM，教师通过摘要指令处理响应 token 后均值池化作为目标嵌入，学生仅从查询预测该目标。重构目标是否仍需保留（用于可解释性而非嵌入质量）是开放问题。

潜在链式推理。将数百响应 token 压缩为 10 个可解码潜在 token 后**链式传递**——将其作为输入附加新 compression token 表示"对响应的响应"，k 步链式推理仅需 k 次前向传播而非数百步自回归解码。

Agent 间潜在通信。多 Agent 系统中以固定长度密集潜在表示替代变长文本 token 通信，保留可解码透明性以支持人类监督。

主要局限：

- 4B/8B 模型检索任务轻微退化，compression token 信息瓶颈限制细粒度信息保留；
- 编码教师必须与学生同家族，跨家族表示空间不兼容；
- 安全性受响应生成器影响——小模型生成质量较差时安全性下降；
- 有监督设置未能充分利用对比学习教师的判别能力。

RetroAgent：通过回顾式双重内在反馈实现从求解到进化

Zhang[1], Liu[2], Zhang, Hu[1], Shao[1*] — [1]Shanghai AI Lab, [2]NUS, 2025

1. 动机与问题定义

基于大语言模型（LLM）的智能体经强化学习（RL）训练后在复杂交互任务中展现出强大潜力，但标准 RL 范式偏重**静态问题求解**而非**持续适应**，存在两个核心缺陷：

- **探索不足**：智能体偏向利用（exploitation），易收敛于次优策略而忽视多样化替代方案；
- **知识隐式化**：学习到的知识隐含于参数中，无法显式检索过往经验以指导当前决策，导致学习效率低、泛化脆弱。

现有工作沿两条独立路线应对：一是通过 meta-RL（LAMER）或熵调制策略梯度（EMPG）促进探索；二是通过记忆机制（MemRL、EvolveR、SkillRL）存储原始轨迹或提炼技能。但这些方法**孤立地**解决上述两个局限，未能弥合问题求解与持续适应之间的鸿沟。

RetroAgent 是首个统一框架，通过**事后回顾式自我反思机制**产生**双重内在反馈**——数值信号与语言信号协同驱动策略优化，使智能体从"解决问题"进化为"持续适应"。

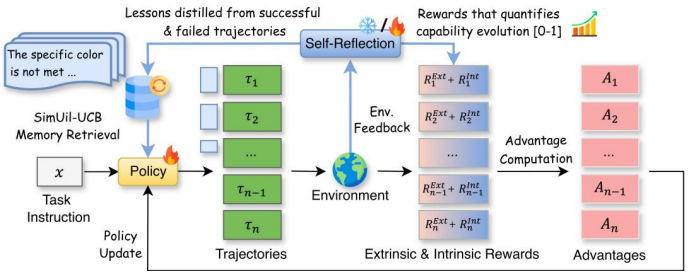


图 1：*RetroAgent* 框架概览。每轮结束后，自我反思机制分析轨迹产生双重内在反馈：内在数值反馈量化相对历史的子任务增量完成度；内在语言反馈将可复用经验提炼至记忆缓冲区，通过 *SimUtil-UCB* 策略检索。

2. 问题建模

将 LLM 智能体的多轮交互建模为 MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$。状态 $s_t = (o_0, a_0, \ldots, a_{t-1}, o_t)$ 为历史观测-动作拼接。外在奖励稀疏——仅在回合结束时给出终端奖励 R^{ext}，均匀分配至各步。RetroAgent 的复合目标为：

$$\mathcal{I}_{\text{RetroAgent}}(\theta) = \mathbb{E}_{\tau \sim \Pi_\theta(\cdot|x) \times P}\left[\sum_{t=0}^{T-1} \gamma^t \left(R^{\text{ext}} + R^{\text{int}}\right)\right],$$

其中 $\Pi_\theta(\cdot|x)$ 为基础策略 $\pi_\theta(\cdot|x)$ 与记忆增强策略 $\pi_\theta(\cdot|f_{\text{memory}}(x, B))$ 的混合分布。

2.1 自我反思机制

每轮结束后，反思函数 $z = f_{\text{reflect}}(\tau)$ 产生三元组 $z = (\phi_{(x,\tau)}, c, m)$：(i) 潜力分数 $\phi_{(x,\tau)} \in [0, 1]$（子任务完成率估计）；(ii) 成功预测 $c \in \{\text{success}, \text{failure}\}$；(iii) 自然语言回顾教训 m。

In-Context 变体：采用**成对归纳**——输入对比参考轨迹 τ_{ref}（与当前回合结果相反）和外在结果指示符 I^{ext}，通过对比成功/失败轨迹更精准地隔离行为优劣。

RL-Trained 变体：引入反思奖励 $R^{\text{reflect}} := R^{\text{ext}} \cdot \mathbb{1}(c = I^{\text{ext}})$，使反思能力与决策策略**联合优化**。复合目标增加自我反思项：

$$\mathcal{I}(\theta) = \underbrace{\mathbb{E}\left[\sum_t \gamma^t (R^{\text{ext}} + R^{\text{int}})\right]}_{\text{决策}} + \underbrace{\lambda_{\text{reflect}} \cdot \mathbb{E}_{z \sim \varphi_\theta(\cdot|\tau)}[R^{\text{reflect}}]}_{\text{反思}}.$$

3. 内在数值反馈：能力进化奖励

潜力分数 $\phi_{(x,\tau)}$ 转化为**能力进化奖励** R^{int}，捕捉相对历史的子任务增量完成。为每个任务 x 维护历史基线 Φ_x（所有先前迭代中最高的组均成功率）。在第 k 轮：

$$R_k^{\text{int}} := \max\left(0, \phi_{(x,\tau),k} - \Phi_x\right),$$

$$\Phi_x := \max\left(\Phi_x, \bar{I}_k^{\text{ext}}\right), \quad \bar{I}_k^{\text{ext}} = \frac{1}{N}\sum_{j=1}^{N} I_k^{\text{ext}(j)}.$$

Φ_x 单调非递减且锚定于已证明的性能，要求潜力分数超越历史最优才获得内在奖励，促进一致的策略改进并防止孤立的不可复现成功主导优化。

4. 内在语言反馈：记忆检索

4.1 反思记忆缓冲区

维护持久缓冲 $B = \{b_i\}$，每条 $b_i = (x_i, m_i, \tau_i, u_i, n_i, d_i)$，包含任务指令、教训、轨迹、效用分数、检索次数和结果标记。使用 all-MiniLM-L6-v2 冻结编码器映射至共享嵌入空间。

4.2 SimUtil-UCB 检索策略

综合三个准则选取 top-k 教训：

准则 1——语义相关性：余弦相似度 $s_{\text{rel}}(x, x_i) = \frac{\mathcal{E}(x) \cdot \mathbf{v}_i}{\|\mathcal{E}(x)\| \|\mathbf{v}_i\|}$，阈值 < 0.4 的候选直接丢弃。

准则 2——反思效用：效用分数初始化为 0.5，每次检索后按指数移动平均更新：$u_i := (1 - \beta_{\text{util}})u_i + \beta_{\text{util}}\hat{u}_t$。

准则 3——探索覆盖：UCB 公式平衡利用与探索：$u_{\text{UCB}}^{(i)} := u_i + \kappa\sqrt{\frac{\ln N}{n_i}}$，$\kappa = 1.0$。

综合得分：$S(b_i|x) := \alpha \cdot s_{\text{rel}}(x, x_i) + (1 - \alpha) \cdot u_{\text{UCB}}^{(i)}$，$\alpha$ 控制相关性与效用的权衡。

4.3 策略优化

每个任务 x 生成 N 条轨迹：前 $N/2$ 从基础策略采样，后 $N/2$ 从记忆增强策略采样——兼顾经验利用与独立探索。决策目标采用 GRPO 的 clipped surrogate + KL 正则化；反思目标采用 REINFORCE。

5. 实验结果

5.1 主实验

在 Qwen-2.5-7B-Instruct 上跨四个任务评估（三次独立运行均值，$p < 0.01$）：

任务	RetroAgent vs GRPO 基线
ALFWorld 成功率	95.6% vs 77.3% (+18.3%)
WebShop 成功率	82.3% vs 66.9% (+15.4%)
Sokoban 成功率	38.3% vs 11.2% (+27.1%)
MineSweeper 成功率	48.2% vs 39.3% (+8.9%)

与最强基线对比：WebShop 超越 GiGPO/SkillRL 约 +6.1–6.2%；Sokoban 超越 GiGPO +16.4%。RetroAgent 显著优于所有记忆增强 RL 框架（MemRL 9.2%、EvolveR 17.6%、SimpleMem+GRPO 46.9%），证实数值 + 语言双重信号优于单一语言指导。同时大幅超越 GRPO w/ EMPG（仅用数值内在反馈），验证双重反馈的互补性。

5.2 测试时适应与泛化

Discovery@k 指标：RetroAgent 在三次尝试内达到近乎完美的发现率——WebShop ID：82.3% → 99.0%；ALFWorld OOD：92.9% → 100.0%，始终优于 LAMER。

记忆内化：移除测试时记忆检索仅导致 Discovery@1 轻微下降（78.9% → 76.8%），Discovery@3 完全保持，表明双重反馈的收益已被**吸收进策略权重**。

困难任务鲁棒性：MineSweeper 地雷数从训练的 3 增至 4–5，RetroAgent 在所有难度下一致优于全部基线，展现优雅降级。

5.3 自我反思分析

成对归纳优于单轨迹归纳：与 GPT-4o oracle 的子任务完成分数相关性更高；幻觉率更低（失败轨迹 3.8% vs 8.8%）；高效用教训比例更高（20.1% vs 12.9%）。下游 GRPO 增强效果：72.9% vs 70.3%。

半组增强优于全组：75.3% vs 72.9%——保留无增强探索对防止过早收敛至关重要。

5.4 数值反馈消融

折扣回报改进 GRPO +8.7 任务分 / +7.8 成功率；叠加能力进化奖励进一步提升至 88.2% / 79.7%；能力进化奖励优于进度引导奖励（用 I^{ext} 替代 ϕ），证实自我评估的潜力分数提供比二元外在结果更丰富的塑形信号。

5.5 语言反馈消融（检索策略）

检索策略	WebShop 成功率
GRPO + 折扣回报（无记忆）	74.7%
+ 相似度检索	70.1%（退化!）
+ 相似度 & 效用检索	69.5%（退化!）
+ SimUtil-UCB	**78.6**%

关键发现：仅用相似度/效用检索配合折扣回报反而**性能退化**（折扣回报可能放大低质量记忆引导的探索行为），而 SimUtil-UCB 通过 UCB 探索奖励有效分散记忆访问——大部分实例约被访问 5 次（vs. 相似度检索中窄子集超 15 次），避免过度依赖语义相似但可能强化次优行为的记忆。

5.6 双重反馈组合

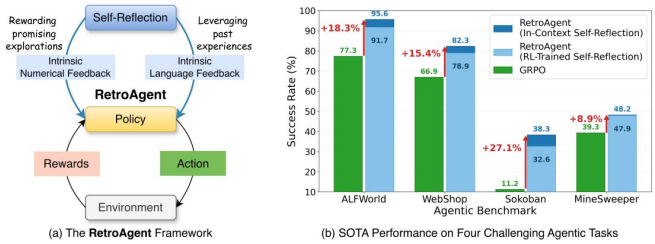


图 2：*RetroAgent* 概览与四任务 *SOTA* 性能对比。

组合双重反馈在 WebShop 上达到 82.3%(RL-Trained),+3% 优于任一单独反馈。RL-Trained 变体（单轨迹归纳）反思准确率在训练中保持稳定并持续上升，而 In-Context 变体准确率随策略改进而**稳步下降**。

Vendi Score 多样性分析：所有内在反馈方法的成功轨迹多样性均高于 GRPO 基线（1.85）。RL-Trained RetroAgent 达到最高 2.20，验证内在反馈确实鼓励有价值的探索。

5.7 训练效率

RetroAgent 总训练时间虽多于 GRPO，但到达基线峰值性能的速度更快：In-Context 变体在第 65 步匹配 GRPO 峰值（**时间减少 46%**），RL-Trained 在第 73 步匹配（**减少 32%**）。

5.8 跨模型泛化

Llama-3.1-8B-Instruct 上一致验证：RetroAgent（In-Context）在 ALFWorld 93.1%（vs GRPO 72.7%）、Sokoban 39.1%（vs 12.2%）、MineSweeper 52.3%（vs 42.4%）；RL-Trained 在 WebShop 达 80.5%（vs 67.6%）、MineSweeper 59.9%。

6. 讨论与局限

- **核心贡献**：首个统一数值 + 语言内在反馈的在线 RL 框架；SimUtil-UCB 检索策略有效平衡相关性、效用与探索；四个多样化任务上全面 SOTA；
- **相关性-效用权衡**：$\alpha = 0.3$（偏重效用）优于 $\alpha = 0.7$（偏重相关性），强调不应仅依赖语义相似度；
- **RL-Trained 变体局限**：在 Llama 上 ALFWorld/Sokoban 反思目标与决策目标存在干扰，联合优化的多目标平衡策略有待改进；
- **In-Context 变体局限**：双通道探索信号可能相互干扰，导致略低于单一数值反馈（WebShop 78.9% vs 79.7%）；
- **未来方向**：更有效的多目标优化策略、扩展至多智能体与开放式环境。

CodePercept：基于代码锚定的 MLLM 视觉 STEM 感知增强

Guan[1], Yang[2], Wan[2], Yang[2], Guo[3], Hu[1], Luo[4], Chen[5], Jiang[5], Wang[2], Shen[1], Lin[2], Yang[1] — [1]SJTU, [2]Alibaba Qwen, [3]BIT, [4]THU, [5]ZJU, 2025

1. 核心动机：感知是 STEM 视觉推理的真正瓶颈

当 MLLM 在 STEM 视觉推理中失败时，根本问题是：**感知不足还是推理受限？**本文通过系统性的**缩放分析**（scaling analysis）回答了这一问题。作者将 STEM 视觉推理解耦为两个阶段：视觉感知（image→caption）和推理（caption→answer），分别独立缩放各组件（使用 Qwen3-VL-Thinking 4B/8B/32B 模型），固定另一组件不变，在 MathVision 数据集上评估。

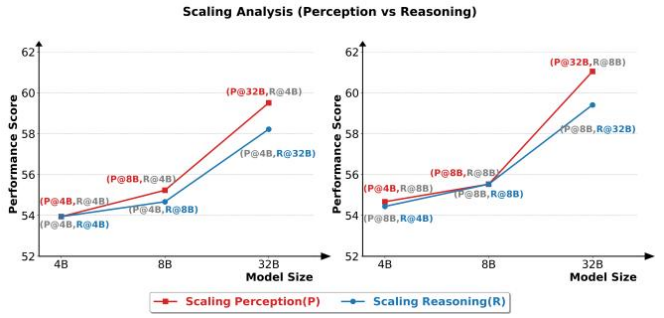


图 *1*：缩放分析。蓝线：固定感知缩放推理；红线：固定推理缩放感知。两组实验均表明缩放感知始终优于缩放推理。

关键发现：无论感知固定在 4B 还是 8B，**缩放感知始终带来更大的性能提升**。这一实证揭示：感知是当前 STEM 视觉推理的真正杠杆。

直觉方案——利用 GPT/Gemini 等高级 MLLM 知识蒸馏生成描述性 caption——面临两个致命缺陷：(1) 教师模型存在**幻觉**，尤其在空间位置、数量关系和元素交互方面；(2) 许多 STEM 图像存在**描述性失语**（descriptive aphasia），其复杂空间关系和精确数值无法被自然语言完整捕获。

本文提出范式转换：**以可执行代码作为感知的锚定介质**。核心原理简洁有力——只有当模型实现完整的视觉理解时，才能生成准确重建原始图像的代码。

2. 系统架构与方法论

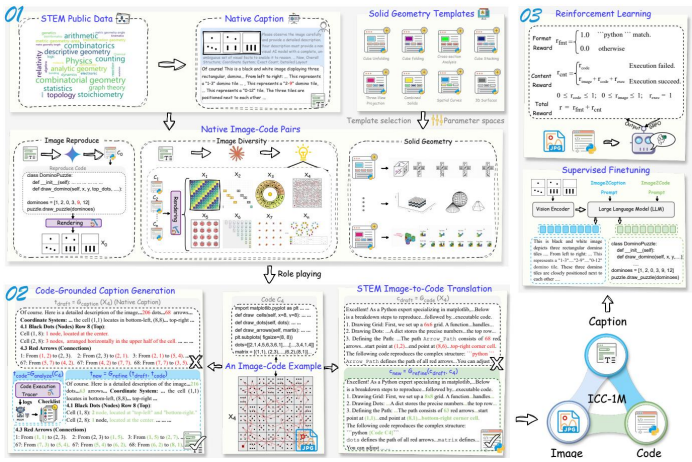


图 *2*：*CodePercept* 完整流程。*Part01*：三条并行流水线构建图像-代码对；*Part02*：两个代码锚定训练任务；*Part03*：*ICC-1M* 数据集与 *SFT+RL* 两阶段训练。

2.1 图像-代码对构建（Part 01）

给定种子图像集 $\mathcal{X}$，通过三条并行流水线扩展为大规模图像-代码数据集：

$$\{(\mathbf{x},\mathbf{c}) \mid \mathbf{c}\in \mathbb{F}_{\mathrm{IR}}(\mathcal{X})\cup \mathbb{F}_{\mathrm{ID}}(\mathcal{X})\cup \mathbb{F}_{\mathrm{SG}}(\mathcal{T}_{\mathrm{geo}}),\ \mathbf{x}\sim \mathcal{R}(\mathbf{c})\}$$

1）图像复现 $\mathbb{F}_{\mathrm{IR}}$**：**对每张 $\mathbf{I}\in\mathcal{X}$，分两阶段生成代码：$\mathbf{c}=\mathrm{G}_{\mathrm{code}}(\mathbf{I},\mathrm{G}_{\mathrm{caption}}(\mathbf{I}))$。先由 MLLM 生成丰富文本描述，再结合图像和描述生成可执行 Python 代码。两阶段策略使模型先显式理解视觉内容再生成代码，提升准确性与可解释性。

2）图像多样化 $\mathbb{F}_{\mathrm{ID}}$**：**基于关键洞察——STEM 图像的底层原理可被系统性抽象并跨不同视觉上下文重新实例化。对每张 $\mathbf{I}$，通过组合式抽象-实例化过程生成 K 个多样代码变体：$[\mathbf{c}_1,\ldots,\mathbf{c}_K]=\mathrm{G}_{\mathrm{code}}(\mathbf{I},\mathrm{G}_{\mathrm{principle}}(\mathbf{I}))$。例如，从多米诺逻辑拼图种子图像出发，可生成环形多米诺轮、三角形排列、网格连接图等，每种均保持 STEM 严谨性的同时引入结构新颖性。

3）立体几何合成 $\mathbb{F}_{\mathrm{SG}}$**：**前两条流水线在立体几何代码生成上存在根本限制——需要精确配置空间关系，当前 LLM/MLLM 在此能力不足。为此构建基于代码模板的合成流水线：定义模板集合 $\mathcal{T}_{\mathrm{geo}}=\{\tilde{\mathbf{c}}_i\}_{i=1}^{M}$，每个模板通过参数空间 Θ_i 控制立方体排列、视角和空间配置等属性，$\mathcal{C}_{\mathrm{geo}}=\{\mathbf{c}_i \mid \mathbf{c}_i=\tilde{\mathbf{c}}_i(\theta);\ \theta\in\Theta_i\}$。

4）统一质量控制：三级复合过滤策略——Q_{I}（图像质量）、Q_{C}（代码质量）、Q_{IC}（图像-代码一致性），均由 SOTA MLLM 配合专业提示评估。仅保留同时满足三项标准的样本。

2.2 代码锚定的 Caption 生成（Sec. 3.2）

直接视觉-文本生成中，复杂 STEM 关系和精确数值易产生幻觉。本文提出利用 ground-truth 代码 $\mathbf{c}$ 作为可靠介质生成准确描述：

$$\mathbf{t}_{\mathrm{new}}=\mathrm{G}_{\mathrm{refine}}(\mathrm{G}_{\mathrm{caption}}(\mathbf{x}),\ \mathrm{G}_{\mathrm{analyze}}(\mathbf{c},\xi(\mathbf{c})))$$

步骤 1）原生 Caption：$\mathbf{t}_{\mathrm{draft}}=\mathrm{G}_{\mathrm{caption}}(\mathbf{x})$，直接描述图像，语言流畅但含事实错误。

步骤 2）代码分析：利用代码本身作为 ground truth 提取可靠视觉事实。关键创新是引入**执行追踪器** $\xi(\mathbf{c})$——在受控环境中执行代码，系统性记录几何精度（精确坐标、维度、空间关系）、数量属性（确切计数、RGB 值）、渲染语义（z-order 层级、变换矩阵）以及 STEM 参数与视觉表现的映射。$\mathbf{t}_{\mathrm{code}}=\mathrm{G}_{\mathrm{analyze}}(\mathbf{c},\xi(\mathbf{c}))$。

步骤 3）代码锚定 Caption：保真重写——$\mathbf{t}_{\mathrm{new}}=\mathrm{G}_{\mathrm{refine}}(\mathbf{t}_{\mathrm{draft}},\mathbf{t}_{\mathrm{code}})$，对 $\mathbf{t}_{\mathrm{draft}}$ 进行外科式编辑：纠正事实错误、用精确计数替换模糊量词、补充遗漏的感知显著内容，同时保留原始句法结构和描述流畅性。

2.3 STEM 图像到代码翻译（Sec. 3.3）

代码提供根本不同于自然语言的视觉描述模态——以编程构件定义视觉内容，捕获几何关系、数学约束和结构细节。构建**解释性代码** $\mathbf{c}_{\mathrm{new}}=\mathrm{G}_{\mathrm{refine}}(\mathrm{G}_{\mathrm{code}}(\mathbf{x}),\mathbf{c})$：先由 MLLM 直接从图像生成带有教学性注释的草稿 $\mathbf{c}_{\mathrm{draft}}$，再利用验证过的 ground-truth 代码 $\mathbf{c}$ 进行代码锚定精炼，保留解释性结构同时确保代码正确性。

2.4 两阶段后训练策略

训练数据形式化为图像-caption-代码三元组 $\mathcal{D}_{\mathrm{train}}=\{(\mathbf{x}^{(i)},\mathbf{t}_{\mathrm{new}}^{(i)},\mathbf{c}_{\mathrm{new}}^{(i)})\}_{i=1}^{N}$。

阶段 1：SFT（CodePercept-S1）：基于 Qwen3-VL 系列，联合优化图像描述 $(\mathbf{x},\mathbf{t}_{\mathrm{new}})$ 和图像到代码翻译 $(\mathbf{x},\mathbf{c}_{\mathrm{new}})$ 两个任务。Caption 提供代码生成的语义上下文，代码通过其精确可执行特性反过来提升 caption 准确度，形成**互补增强**。

阶段 2：RL（CodePercept-R1）：专门对代码生成应用 GRPO 强化学习。定义两类奖励：格式奖励 r_{fmt}（正则验证 Python 代码块格式）和内容奖励 r_{cnt}（含执行成功奖励 r_{exec}、GPT-4o 评估的代码语义等价分数 r_{code}、渲染图像与原图的视觉相似度 r_{image}）。总奖励 $r=r_{\mathrm{fmt}}+r_{\mathrm{cnt}}$。GRPO 目标函数：

$$\mathcal{J}(\theta)=\mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\min\left(\frac{\pi_\theta(o_i|q)}{\pi_{\theta_{\mathrm{old}}}(o_i|q)}A_i,\ \mathrm{clip}(\cdot,1-\epsilon,1+\epsilon)A_i\right)-\beta D_{\mathrm{KL}}\right]$$

其中优势函数 $A_i=(r_i-\mathrm{mean}(\{r_j\}))/\mathrm{std}(\{r_j\})$。

3. STEM2Code-Eval 基准

提出首个**直接评估 STEM 视觉感知**的确定性可验证基准——要求模型生成可执行 Python 代码忠实重建原始图像。

包含 1000 张手工标注图像，来源于 6 个 VQA-based STEM 基准（MathVista, MathVision, MathVerse, DynaMath, We-Math, LogicVista）。

构建流程：(1) Gemini2.5-Pro 生成详细 caption → Claude-Opus 4.1 合成 matplotlib 代码；(2) 迭代代码智能体精炼（渲染 → 错误修复 → 相似度评分 → 改进建议），直至分数 > 90% 或达 10 轮；(3) 候选筛选（按相似度分数和迭代次数排序取 top 3k）；(4) 10 名专家标注员按 style/content/functionality 三维度 5 分制评分，取 top 1000 并进行像素级人工微调。

三项评估指标：Image Score（视觉相似度）、Code Score（代码质量与正确性）、Exec Rate（执行成功率）。

4. 核心实验结果

4.1 问题求解感知评估（Captioner-Solver 范式）

模型（Captioner）	6 基准平均（%）
Gemini2.5-Pro	70.53
GPT5-Thinking	62.23
Claude-Opus 4.1-Thinking	60.72
Qwen3-VL-4B-Instruct	59.29
CodePercept-4B-S1	**62.07** (+2.8)
Qwen3-VL-8B-Instruct	60.36
CodePercept-8B-S1	**63.32** (+3.0)
Qwen3-VL-32B-Instruct	64.63
CodePercept-32B-S1	**67.30** (+2.7)

以 Qwen3-30A3-Thinking 为固定 Solver。CodePercept 在所有规模上均实现显著提升。使用更强的 Qwen3-235A22-Thinking Solver 时，CodePercept-8B-S1 提升达 +3.4%，CodePercept-32B-S1 提升达 +3.6%（71.13 vs 67.50）。值得注意的是，CodePercept-8B-S1 超越了 Qwen2.5-VL-72B（+6.2%），接近 Claude-Opus 4.1-Thinking 和 GPT5-Thinking 等前沿模型。

4.2 图像重建感知评估（STEM2Code-Eval）

模型	Avg Score	Exec Rate
Gemini2.5-Pro-Thinking	72.15	91.7%
GPT5-Thinking	64.98	96.6%
Qwen3-VL-4B-Instruct	25.49	79.4%
CodePercept-4B-S1	40.78 (+15.3)	80.7%
CodePercept-4B-R1	**46.52** (+21.0)	91.3%
Qwen3-VL-8B-Instruct	28.41	85.3%
CodePercept-8B-S1	45.66 (+17.3)	87.6%
CodePercept-8B-R1	**48.65** (+20.2)	93.4%
Qwen3-VL-32B-Instruct	38.42	81.8%
CodePercept-32B-S1	59.07 (+20.7)	93.0%
CodePercept-32B-R1	**65.75** (+27.3)	95.9%

CodePercept-S1 在直接感知评估上实现了巨大提升（4B: +15.3，8B: +17.3，32B: +20.7）。经 RL 进一步优化后，CodePercept-4B-R1 达 46.52，CodePercept-8B-R1 达 48.65，均**超越了 Seed1.6-Vision 和 Qwen3-VL-Plus** 等超大模型（后者约 41–43 分）。CodePercept-32B-R1 达 65.75，接近 GPT5-Thinking（64.98）。

5. 消融实验

1) 三条流水线逐步添加：IR-CodeCap（60.91）→ +ID-CodeCap（62.15）→ +SG-CodeCap（62.75），基线 Qwen3-VL-8B 为 60.36。Image Diversity 流水线贡献最大（+1.24），验证了多样化 STEM 图像提供更强训练信号。

2) 代码锚定 Caption vs 原生 Caption：在同一 ICC-1M 数据上，NativeCap（直接由 Gemini2.5-Pro 生成）平均 60.78，CodeCap（代码锚定）62.75，**显著提升 +2.0**。验证了通过从可执行代码中提取视觉事实，CodeCap 有效减少了数值和几何细节上的幻觉。

3) 代码作为互补模态：在 CodeCap 基础上添加 ImCode（图像到代码翻译任务），性能进一步提升至 63.32（+0.6）。证明代码作为结构化描述与自然语言 caption 互补——caption 为代码生成提供语义上下文，代码通过精确可执行表示克服自然语言描述复杂数学视觉的固有局限。

6. 与相关工作的对比

MLLM for STEM 领域现有工作主要聚焦推理增强：冷启动思维数据策划、基于 RL 的方法（reward 机制）、单模态思维数据迁移。本文首次揭示**感知而非推理**是 STEM 的主要瓶颈。

STEM 感知评估方面，OmniCaptioner 等采用两阶段评估（caption→LLM 求解），但仅测量问题相关信息提取而非全面视觉理解。本文 STEM2Code-Eval 要求生成可执行重建代码，提供确定性可验证的感知评估，且与 UI/Chart/SVG 等领域特定代码生成任务不同，同时服务于评估基准和训练数据构建的双重目的。

7. 局限性与未来方向

- 当前基于 matplotlib 渲染，覆盖的 STEM 视觉类型有限（如化学分子结构、电路图等需要专业库支持）；
- RL 阶段依赖 GPT-4o 作为奖励模型，成本较高且引入外部模型偏差；
- 立体几何合成仍依赖人工模板设计，自动化程度有待提升；
- 代码锚定方法的有效性依赖于种子数据集的质量和多样性；
- 未来可将代码锚定范式扩展至更多 STEM 子领域，并探索无需外部奖励模型的自评估 RL 策略。

PRISM-Δ：基于差分子空间引导的大语言模型 Prompt 高亮方法

Ge[1], Liu[1*], Wang[2], Liu[3], Bi[1], Mei[1], Yao[1], Guo[1], Cheng[1] — [1] 中科院计算所, [2]UC Merced, [3] 北大, 2025

1. 动机与问题定义

Prompt highlighting（提示高亮）旨在引导大语言模型在生成时优先关注用户指定的文本片段。典型场景包括：信息冲突时模型应优先采信新提供的事实而非参数记忆，长上下文检索中答案位于中间段落时模型表现显著下降（Lost-in-the-Middle 现象）。

现有方法存在根本性局限——**仅操作路由通道**（Key 表示），忽略内容通道（Value 表示）：

- **PASTA**：事后修改注意力分数矩阵，与 FlashAttention 不兼容，额外开销 +1.03s / +23GB；
- **SPA**：在 logit 层面锚定，需多次前向传播，额外 +5.32s；
- **SEKA**：通过独立 SVD 分解编辑 Key 向量，开销极低但使用硬阈值二值化头权重，且仅操作 Key。

核心观察：作者提取五个模型在对比条件下的 Key 和 Value 表示，发现 Value 偏移量与 Key 可比，约**半数注意力头**存在显著 Value 通道信号。两通道信号弱相关（r=0.342），且在网络深度上互补——Key 信号峰值在中间层（L13–24），Value 信号峰值在后层（L25–36）。Key-only 方法遗漏了大量有用信号。

2. 系统架构与方法

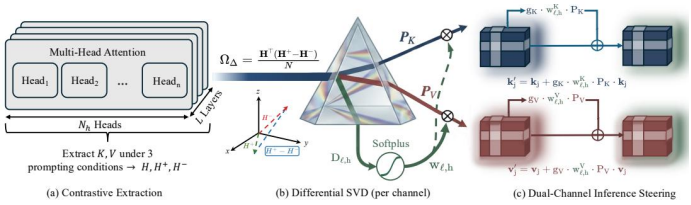


图 1：*PRISM-Δ* 概览。*SVD* 分解 Ω_Δ 得到每头投影（P_K, P_V）和重要性权重 $w_{\ell,h}$，推理时同时引导 *Key* 和 *Value* 通道。

2.1 双通道注意力视角

标准多头注意力中，输出由路由通道和内容通道联合决定：

$$\text{output}_i = \sum_j \underbrace{\text{softmax}\left(\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d}}\right)}_{\alpha_{ij}\text{:路由}} \cdot \underbrace{\mathbf{v}_j}_{\text{内容}}$$

若同时扰动高亮 token 的两通道，输出变化分解为三项：

$$\text{output}'_i = \text{output}_i + \underbrace{\sum_j \Delta\alpha_{ij} \cdot \mathbf{v}_j}_{\text{路由增益}} + \underbrace{\sum_j \alpha_{ij} \cdot \Delta\mathbf{v}_j}_{\text{内容增益}} + \underbrace{\sum_j \Delta\alpha_{ij} \cdot \Delta\mathbf{v}_j}_{\text{交叉增益}}$$

现有方法仅捕获路由增益；PRISM-ΔV 捕获全部三项。

2.2 差分交叉协方差分解

构造合成 QA 三元组，在答案 token 位置提取三种条件下的表示：$\mathbf{H}$（中性，仅上下文）、$\mathbf{H}^+$（正向，上下文 + 相关问题）、$\mathbf{H}^-$（负向，上下文 + 无关问题）。定义无中心交叉协方差 $\Omega^+ = \mathbf{H}^\top \mathbf{H}^+ / N$，其顶部奇异方向可能包含在两种条件下同等变化的**共享方向**。为此分解**差分交叉协方差**：

$$\Omega_\Delta = \mathbf{H}^\top(\mathbf{H}^+ - \mathbf{H}^-)/N = \Omega^+ - \Omega^-$$

命题 1（差分方向的判别最优性）：令 $\Omega_\Delta = U_\Delta \Sigma_\Delta V_\Delta^\top$ 为 SVD 分解，则：(a) **最大判别能量**：前 k 个左奇异向量 $\{u_1, \ldots, u_k\}$ 求解 $\max_{U^\top U = I} \|U^\top \Omega_\Delta\|_F^2$，即捕获正负条件间交叉协方差差异最大的 k 维子空间（Eckart–Young 定理）；(b) **自动消除共享方向**：若 $\Omega^+ \mathbf{u}_s = \Omega^- \mathbf{u}_s$，则 $\Omega_\Delta \mathbf{u}_s = \mathbf{0}$，共享方向对差分投影贡献为零。该命题无需分布假设，对任意有限样本集成立。

投影矩阵 $P = U_\Delta[:, :k] \cdot U_\Delta[:, :k]^\top$，其中 k 选取使累积奇异值比 $\sum_{i=1}^k \sigma_i / \sum_{i=1}^d \sigma_i \geq \gamma$。实验表明独立投影 P^+（秩 89）和 P^-（秩 39）存在重叠子空间（$\text{tr}(P^+P^-)$≈1.31），而差分投影 P_Δ 直接定位判别子空间。

2.3 Softplus 头重要性加权

SVD 过程自然产出每头判别度——范数差 $D_{\ell,h} = \frac{1}{N}\sum_i \|\mathbf{r}_i^+ - \mathbf{r}_i^-\|_2$，量化该头在正负条件下的输出偏移。通过 softplus 映射为连续权重：

$$w_{\ell,h} = \text{softplus}(D_{\ell,h} - \delta_{\min}) = \log(1 + \exp(D_{\ell,h} - \delta_{\min}))$$

相比 SEKA 的硬阈值（将 108/288 个头完全关闭），softplus 平滑插值——强头全激活，弱头低权重贡献，噪声头近零。

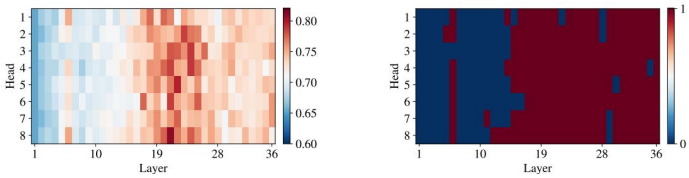


图 2：头权重热力图（*36* 层 ×*8* 头）。左：*PRISM-Δ softplus* 权重呈连续渐变（$[0.654, 0.808]$）。右：*SEKA* 硬阈值创建二值分割，关闭 *108* 头（含 *90%* 早期层头）。

2.4 双通道引导

对 Key 和 Value 空间分别应用子空间学习：

$$\Omega_\Delta^K = \mathbf{H}_K^\top(\mathbf{H}_K^+ - \mathbf{H}_K^-)/N \xrightarrow{\text{SVD}} P_K, w_K$$

$$\Omega_\Delta^V = \mathbf{H}_V^\top(\mathbf{H}_V^+ - \mathbf{H}_V^-)/N \xrightarrow{\text{SVD}} P_V, w_V$$

推理时，对每个高亮 token $j \in \mathcal{S}$，同时编辑：

$$\mathbf{k}'_j = \mathbf{k}_j + g_K \cdot w_{\ell,h}^K \cdot P_K \cdot \mathbf{k}_j, \quad \mathbf{v}'_j = \mathbf{v}_j + g_V \cdot w_{\ell,h}^V \cdot P_V \cdot \mathbf{v}_j$$

几何解释：变换 $(I + g \cdot w \cdot P)\mathbf{x}$ 将 $\mathbf{x}$ 在学习子空间内的分量放大 $(1 + g \cdot w)$ 倍，正交分量不变。与均匀放大不同，每头放大因子由判别度 $w_{\ell,h}$ 个别调节；与 logit 偏置不同，Key 缩放产生**查询依赖**的注意力提升。

两种实例化：PRISM-Δ（g_V=0，仅路由通道）和 PRISM-ΔV（g_K>0, g_V>0，双通道）。

3. 实验设置

基准任务：BiasBios（职业预测，指标：准确率/流畅度/一致性）、CounterFact（知识冲突，指标：有效性/释义）、Pronoun Change（代词替换，指标：P. Score/All-changed）、Lost-in-the-Middle（30 段长上下文检索，指标：精确匹配）。

模型：QWEN3-4B/8B/14B-Base 和 GEMMA3-4B/12B-PT，涵盖两种架构三种规模。

基线：Original、**-marked、PASTA、SPA、SEKA。投影由 100 个 GPT-4o 生成的合成对比 QA 对离线构造，单 GPU 约 3–8 分钟。

4. 核心实验结果

4.1 短上下文基准

任务/模型	指标	SEKA	PRISM-Δ	ΔV
BB/Q3-4B	Acc	90.92	**92.38**	92.36
BB/Q3-8B	Acc	88.74	**89.62**	89.12
BB/Q3-14B	Acc	90.28	**91.68**	91.20
CF/Q3-8B	Eff	99.08	**99.24**	98.46
PC/G3-4B	P.Sc	81.53	89.08	**90.16**
PC/Q3-14B	P.Sc	98.66	**99.66**	99.66

在 20 个模型 × 基准配置中，PRISM-Δ（含 ΔV）在 **19/20** 上匹配或超越最佳现有方法，相对增益最高达 **+10.6%**（Pronoun Change, GEMMA3-4B）。统计可靠性高：五组投影子集标准差仅 0.05–0.15%，14/15 模型 × 基准单元 PRISM-Δ 匹配或超越 SEKA（p<0.001）。

4.2 长上下文检索（Lost-in-the-Middle）

模型	Vanilla	SEKA	PRISM-Δ	ΔV
Q3-4B	45.47	49.14	**51.43**	50.57
Q3-8B	46.69	50.29	**52.71**	51.86
Q3-14B	57.13	60.86	**62.57**	62.00
G3-4B	44.36	47.71	49.29	**50.14**
G3-12B	51.64	55.57	54.43	**55.57**

30 段输入的中间区域引导中，PRISM-Δ 在全部 5 个模型上匹配或超越 SEKA，Q3-8B 上相对增益达 **+4.8%**。

4.3 效率分析

PRISM-Δ 延迟 +0.30s（1.26×），内存仅 +0.02GB；ΔV 再增 +0.02s。两者均**兼容 FlashAttention**。对比：PASTA +1.03s/+23GB，SPA +5.32s。

5. 消融实验

采用 2×2 析因设计（BiasBios, QWEN3-4B）：

- 去除 softplus（均匀 w=1）：准确率降 0.96%；
- 去除差分投影（独立 SVD）：降 0.86%；
- 两者交互呈**强超加性**：单独加差分仅 −0.02%，单独加 softplus 仅 +0.08%，组合却达 +0.94%——差分分解通过降低有效阈值纳入更多头，而 softplus 正是用于降权其中噪声头的关键机制；
- 匹配 $\delta_{\min}$=0.08 时，PRISM-Δ 达最优而 SEKA 下降 1.36%；
- 仅 Value 通道（PRISM-Δ-V）达 82.44%（+3.3% over Vanilla），证实 Value 通道独立携带判别信号。

数据量消融：50 个样本即达 91.92%（仅低于默认 100 个的 0.46%），200 个几乎无额外增益（+0.02%），展现高数据效率。

6. 分析

6.1 判别子空间质量

方向一致性： Ω^+ 方向跨头余弦相似度高达 0.254（3× 随机基线 0.079），确认独立 SVD 被共享结构方向主导；Ω_Δ 方向仅 0.068（低于随机基线），每头提取近乎独立的判别方向。

头选择鲁棒性： 扫描 $\delta_{\min}$ 从 0.06 到 0.20，SEKA 最优至最差降 3.14%，PRISM-Δ 波动仅 0.60%。逐样本分析显示 PRISM-Δ 挽救 154 个 SEKA 错误样本（蓝），仅失去 81 个（红），净增 +73。2:1 不对称性反映对共享特征混淆的系统性修正，符合命题 1(b)。

6.2 双通道贡献分解

Key 通道主导准确率提升（+12.58% over Vanilla），Value 通道保护生成质量。PRISM-Δ 仅产生 SEKA **53%** 的流畅度代价（0.504 vs 0.957）。两个变体在准确率、流畅度、一致性上**同时**超越 SEKA。层级互补：Key 信号中间层均值 0.175，Value 后层均值 0.307，L28 后 Value 超越 Key——功能分工而非冗余。

6.3 跨模型 K/V 互补性

Qwen3 系列 Value 信号随模型增大而增强（K/V 比：1.02 → 0.74 → 0.58）；Gemma3 系列绝对 norm_diff 为 Qwen3 的 3–4×，Key 始终占优（K/V > 1.2）。高信号幅度解释了 Gemma3 需更小 g_K 以避免过度引导。

7. 与 AdaSEKA 对比

AdaSEKA 是 SEKA 的多专家查询自适应扩展（+0.27s, +15.59GB），属不同效率级别。在 15 个模型 × 基准单元中，最佳轻量方法在 **10/15** 上匹配或超越 AdaSEKA，且仅需其 1/9 延迟和可忽略额外内存。AdaSEKA 仅在 Pronoun Change GEMMA3-4B 上优势明显（+3.60%）。

8. 局限与展望

- 最优增益 g_K 因任务和模型而异，需验证集扫描（通常 5–8 个值）；Gemma3 超参偏离 Qwen3 默认值更大；
- GEMMA3-12B 在 Pronoun Change 上需负增益 g_K=−0.30（SEKA 亦有此现象），反映模型默认倾向与对比方向的符号关系；
- 接近饱和的基准（如 CounterFact 98–99%）上绝对增益天然有限；
- Prompt highlighting 赋予用户控制输入优先级的能力，存在滥用风险（如放大误导片段），可通过限制高亮至系统级提示缓解；
- 未来方向：自动 g_K 选择启发式（如按 mean norm_diff 反比缩放）、更多架构验证、与其他 token 级注意力干预的结合。